

IFR (COMP2065) lecture notes

[Search...](#)

Table of Contents

1. [Introduction](#)
2. Propositional Logic
3. Classical Logic
4. Predicate Logic
5. The Booleans
6. The Natural Numbers
7. Lists
8. Trees

1. [Introduction](#)

- 1.1. Interactive proof systems
- 1.2. Lean
- 1.3. Lean and AI

[← IFR \(COMP2065\) lecture notes](#)[2. Propositional Logic →](#)

1. Introduction

These are lecture notes for COMP2065: Introduction to formal reasoning. The main goal is to teach formal logic using an interactive proof system called **Lean**. You will be able to use predicate logic to make precise statements and to verify them using a proof system. This covers both statements in Mathematics and statements about computer programs, e.g. their correctness.

1.1. Interactive proof systems

Interactive proof systems have been used in the past to verify a number of interesting statements, for example the four colour theorem (that every map can be coloured with 4 colour) and the formal correctness of an optimising C compiler. The formal verification of hardware like processors is now quite standard and there is a growing number of software protocols being formally certified. In academia it is now common to accompany a paper in theoretical computer science with a formal proof to make sure that the proofs are correct. Mathematicians are just starting to verify their proofs.

Even if you are not going to use formal verification, it is an important part of a computer science degree to have some acquaintance with formal logic, either to read formal statements made by others or by being able to express facts precisely without ambiguity. I believe that the best way to learn logic these days is by using an interactive proof system because this removes any ambiguity on what constitutes a proof and also you can play with it on your own without the need of a teacher or tutor.

An interactive proof system is not an automatic theorem prover. The burden to develop a correct proof is on you, the interactive proof systems guides you and makes sure that the proof is correct. Having said this, modern proof assistants offer a lot of automatisisation which enables the user not to have to get bogged down in trivial details. However, since our goal is to learn proofs, at least initially we won't use much automatisisation. It is like in Harry Potter, to be allowed to use the more advanced spells you first have to show that you master the basic ones.

1.2. Lean

Many interactive proof systems are based on Type Theory, this is basically a functional programming language with a powerful type system that allows us to express propositions as types and proofs as programs. A well known example is the Rocq system (formerly known as Coq) which was developed (and still is) in France. However, we will use a more recent system which in many respects is similar to Coq, this is Lean which was (and is) developed under the leadership of Leonardo de Moura at Microsoft Research. Leonardo is famous for his work on

automatic theorem proving, he developed the Z3 theorem prover. Lean's goal is to connect automatic and interactive theorem proving. The system is called **Lean** because it only relies on a small core of primitive rules and axioms to make sure that it is itself correct.

[Lean is available for free](#). It will be already installed on the lab machines (I hope). You can use it either via Microsoft's Visual Studio Code or via emacs using lean mode. Yet another way to use lean is via a [browser based version](#) we will also use in the online version of these lecture notes.

Here is an example of a simple proof in Lean: we show that the sum of the first n odd numbers is the square of n . (Actually this example already uses some advanced magic in form of the `ring` tactic).

```
import Mathlib.Tactic
open Nat

def sumN : ℕ → (ℕ → ℕ) → ℕ
| 0, _ => 0
| n+1, f => f n + sumN n f

def nthOdd (i : ℕ) : ℕ :=
  2*i+1

#eval nthOdd 3

#eval sumN 5 nthOdd

theorem oddSum (n : ℕ) : sumN n nthOdd = n^2 := by
  induction n with
  | zero =>
    simp [sumN]
  | succ n ih =>
    simp [sumN]
    calc
      2 * n + 1 + sumN n nthOdd
      = 2 * n + 1 + n^2 := by rw [ih]
      _ = (n + 1)^2 := by ring
```

If you read this online, you can explore the proof and the output by moving the mouse over the listing. You can also copy and paste the code into the [web version of lean](#) and explore or modify it interactively. Alternatively you can use Visual Studio Code.

You can put the cursor in the proof to see what the proof state is and you can evaluate the expressions after `#eval` and maybe change the parameters. You can also change the proof (maybe you have a better one) or do something completely different (but then don't forget to save your work by copy and paste). While you can work using the browser version only, for bigger exercises it may be better to install Lean on your computer.

The Lean community is very active. If you want to know more about Lean and its underlying theory, I recommend the book [Theorem Proving in Lean 4](#) whose online version also uses the web interface. A good place for questions and discussions is the [Lean zulip chat](#) but please don't post your coursework questions anywhere on social networks --- this is considered academic misconduct. And yes, we do have staff members who can read other languages than English.

1.3. Lean and AI

The rise of AI, and in particular of large language models, has changed the role of formal verification. Until recently, formally verifying a mathematical proof or a piece of software was often only possible **in principle**: the effort required was so large that it was rarely attempted in practice. An example is the FLT project, whose goal is to formally verify Fermat's Last Theorem in Lean. Here is the formal statement of the theorem --- notice that it is easy to state but was famously hard to prove:

```
theorem fermatLastTheorem :  
   $\forall n : \mathbb{N}, n > 2 \rightarrow \neg \exists a b c : \mathbb{N}, a > 0 \wedge b > 0 \wedge c > 0 \wedge$   
  sorry  $\square$ 
```

This project has progressed much faster than anticipated, largely thanks to the use of AI.

The same is true for software. Formal verification of real systems was always possible in principle, but because of the immense effort it required it was only ever applied to a very small number of systems. This is now changing: AI is becoming an enabling technology for formal verification.

What does this mean for us? Even if we use AI to help us create formal proofs, we still need to understand the underlying concepts and principles. Hence, while the use of AI in this module, especially as a tutor, is encouraged, we advise against using AI to do your coursework for you --- if you do, you won't learn anything. It is a bit like taking a forklift to the gym to lift weights.



Using AI to do your coursework is like taking a forklift to the gym: the weights get lifted, but your muscles stay small.

← IFR (COMP2065) lecture notes

2. Propositional Logic →

IFR (COMP2065) lecture Search...

Table of Contents

1. Introduction
2. Propositional Logic
3. Classical Logic
4. Predicate Logic
5. The Booleans
6. The Natural Numbers
7. Lists
8. Trees

2. Propositional Logic

- 2.1. What is a proposition?
- 2.2. Our first proof
- 2.3. Using assumptions
- 2.4. Proof terms
- 2.5. Conjunction
- 2.6. The Currying Equivalence
- 2.7. Disjunction
- 2.8. Logic and algebra
- 2.9. True, false and negation
- 2.10. Use have
- 2.11. Summary of tactics

← 1. Introduction

3. Classical Logic →

2. Propositional Logic

2.1. What is a proposition?

A proposition is a definitive statement which we may be able to prove. In Lean we write $P : \text{Prop}$ to express that P is a proposition.

We will later introduce ways to construct interesting propositions i.e. mathematical statements or statements about programs, but in the moment we will use propositional variables instead. We declare in Lean:

```
variable (P Q R : Prop)
```

This means that P Q R are propositional variables which may be substituted by any concrete propositions. In the moment it is helpful to think of them as statements like "The sun is shining" or "We go to the zoo."

We introduce a number of connectives and logical constants to construct propositions:

- Implication ($\rightarrow$), read $P \rightarrow Q$ as **if** P **then** Q .
- Conjunction ($\wedge$), read $P \wedge Q$ as P **and** Q .
- Disjunction ($\vee$), read $P \vee Q$ as P **or** Q . Note that we understand **or** here as inclusive, it is ok that both are true.

- False, read False as **Pigs can fly.**
- True, read True as **It sometimes rains in England.**
- Negation ($\neg$), read $\neg P$ as **not** P .

We define $\neg P$ as $P \rightarrow \text{False}$.

- Equivalence, ($\leftrightarrow$), read $P \leftrightarrow Q$ as P **is equivalent to** Q .

We define $P \leftrightarrow Q$ as $(P \rightarrow Q) \wedge (Q \rightarrow P)$.

As in algebra we use parentheses to group logical expressions. To save parentheses there are a number of conventions:

- Implication and equivalence bind weaker than conjunction and disjunction.

E.g. we read $P \vee Q \rightarrow R$ as $(P \vee Q) \rightarrow R$.

- Implication binds stronger than equivalence.

E.g. we read $P \rightarrow Q \leftrightarrow R$ as $(P \rightarrow Q) \leftrightarrow R$.

- Conjunction binds stronger than disjunction.

E.g. we read $P \wedge Q \vee R$ as $(P \wedge Q) \vee R$.

- Negation binds stronger than all the other connectives.

E.g. we read $\neg P \wedge Q$ as $(\neg P) \wedge Q$.

- Implication is right associative.

E.g. we read $P \rightarrow Q \rightarrow R$ as $P \rightarrow (Q \rightarrow R)$.

This is not a complete specification. If in doubt use parentheses.

We will now discuss how to prove propositions in Lean. If we are proving a statement containing propositional variables then this means that the statement is true for all replacements of the

variables with actual propositions. We say it is a tautology.

Tautologies are sort of useless in everyday conversations because they contain no information. However, for our study of logic they are important because they exhibit the basic figures of reasoning.

2.2. Our first proof

In Lean we write $p : P$ for p proves the proposition P . For our purposes a proof is a sequence of **tactics** affecting the current proof state (the assumptions we have made and the current goal). In Lean 4, a tactic proof starts with `by`, and tactics go on separate lines (no commas).

We start with a very simple tautology $P \rightarrow P$: If P then P . We can illustrate this with the statement **if the sun shines then the sun shines**. Clearly, this sentence contains no information about the weather; it is vacuously true—indeed, a tautology.

Here is how we prove it in Lean:

```
theorem I : P → P := by
  intro h
  exact h
```

We tell Lean that we want to prove a `theorem` (maybe a bit too grandiose a name for this) named `I` (for identity). The actual proof is just two lines, which invoke **tactics**:

- `intro h` means that we are going to prove an implication by assuming the premise (the left hand side) and using this assumption to prove the conclusion (the right hand side). If you look at the html version of this document you can view the proof state by

hovering over the bubble at the end of a line. When you move the cursor after `by` you see that the proof state is:

```
P : Prop
⊢ P → P
```

This means we assume that P is a proposition and want to prove $P \rightarrow P$. The $\vdash$ symbol (pronounced **turnstile**) separates the assumptions and the goal. After `intro h` the proof state is:

```
P : Prop,
h : P
⊢ P
```

This means our goal is now P but we have an additional assumption $h : P$.

- `exact h` We complete the proof by telling Lean that there is an assumption that **exactly** matches the current goal. If you move the cursor to the end of the line you see `All goals completed`. We are done.
- Alternatively we can use `assumption` in the last line which just uses any assumption which fits without us having to tell it which one.

2.3. Using assumptions

Next we are going to prove another tautology: $(P \rightarrow Q) \rightarrow (Q \rightarrow R) \rightarrow P \rightarrow R$. Here is a translation into English:

If if the sun shines then we go to the zoo then if if we go to the zoo then we are happy then if the sun shines then we are happy.

Maybe this already shows why it is better to use formulas to write propositions.

Here is the proof in Lean (I call it **C** for **compose**).

```
theorem C : (P → Q) → (Q → R) → P → 1
  intro p2q
  intro q2r
  intro p
  apply q2r
  apply p2q
  exact p
```

First of all it is useful to remember that $\rightarrow$ associates to the right; putting in the invisible brackets this corresponds to:

$$(P \rightarrow Q) \rightarrow ((Q \rightarrow R) \rightarrow (P \rightarrow R))$$

After the three `intro` we are in the following state:

```
P Q R : Prop,
p2q  : P → Q,
q2r  : Q → R,
p    : P
⊢ R
```

Now we have to **use** an implication. Clearly it is `q2r` which can be of any help because it promises to show `R` given `Q`. Hence once we say `apply q2r` we are left to show `Q`:

```
P Q R : Prop,
p2q  : P → Q,
q2r  : Q → R,
p    : P
⊢ Q
```

The next step is to use `apply p2q` to reduce the goal to `P`, which can be shown using `exact p`.

Note that there are two kinds of steps in these proofs:

- `intro h` means that we are going to prove an implication $P \rightarrow Q$ by assuming P (and we call this assumption h) and then proving Q

with this assumption.

- `apply h` if we have assumed an implication $h : P \rightarrow Q$ and our current goal matches the right hand side we can use this assumption to **reduce** the problem to showing P (whether this is indeed a good idea depends on whether it is actually easier to show P).

The `apply` tactic is a bit more general; it can also be used to handle repeated implications. Here is an example:

```
theorem swap : (P → Q → R) → (Q → P → R)
intro f q p
apply f
exact p
exact q
```

After `intro f q p` (which is a shortcut for writing `intro` three times) we are in the following state:

```
P Q R : Prop
f : P → Q → R
q : Q
p : P
⊢ R
```

Now we can use `f` because its conclusion matches our goal, but we are left with two goals:

```
2 goals
P Q R : Prop
f : P → Q → R
q : Q
p : P
⊢ P
```

```
P Q R : Prop
f : P → Q → R
q : Q
p : P
⊢ Q
```

After completing the first goal with `exact p` it disappears and only one goal is left:

```
P Q R : Prop
f : P → Q → R
q : Q
p : P
⊢ Q
```

which we can quickly eliminate using `exact q`.

2.4. Proof terms

What is a proof? It looks like a proof in Lean is a sequence of tactics. But this is only the surface: the tactics are more like editor commands which **generate** the real proof, which is a **program**. This also explains the syntax `p : P`, reminiscent of the notation for typing `3 :: Int` in Haskell (that Haskell uses `::` instead of `:` is a regrettable historic accident).

We can look at the programs generated from proofs by using the `#print` command in Lean. For example:

`#print I`

```
theorem PropLogicProofs.I :
  ∀ (P : Prop), P → P :=
  fun P h => h
```

and

`#print C`

```
theorem PropLogicProofs.C :
  ∀ (P Q R : Prop), (P → Q) → (Q → R) →
  :=
  fun P Q R p2q q2r p => q2r (p2q p)
```

If you have studied functional programming (e.g. **Haskell**) you should have a **déjà vu**: indeed proofs are **functional programs**. Lean exploits the

propositions as types translation (also known as the **Curry–Howard Equivalence**) and associates to every proposition the type of evidence for this proposition. This means that to see that a proposition holds all we need to do is to find a program in the type associated to it.

Not all Haskell programs correspond to proofs; in particular, general recursion is not permitted in proofs, only certain forms of recursion that always terminate. Also, the Haskell type system isn't expressive enough to be used in a system like Lean: it is fine for propositional logic, but it doesn't cover predicate logic, which we will introduce soon. The functional language on which Lean relies is called **dependent type theory** — more specifically, the **Calculus of Inductive Constructions**.

Type theory is an interesting subject, but we won't be able to say much in this course. If you want to learn more about this, you can attend **Proofs, Programs and Types** (COMP4074), which can also be done in year 3.

2.5. Conjunction

How to prove a conjunction? Easy! To prove $P \wedge Q$ we need to prove both P and Q . This is achieved via the `constructor` tactic. Here is a simple example (and since I don't want to give it a name, I am using `example` instead of `theorem`).

```
example : P → Q → P ∧ Q := by
  intro p q
  constructor
  exact p
  exact q
```

`constructor` turns the goal:

```

P Q : Prop
p : P
q : Q
⊢ P ∧ Q

```

into two goals:

```

2 goals
P Q : Prop
p : P
q : Q
⊢ P

```

```

P Q : Prop
p : P
q : Q
⊢ Q

```

Now what is your next question? Exactly! How do we use a conjunction in an assumption? Here is an example: we show that $\wedge$ is **commutative**.

```

theorem comAnd : P ∧ Q → Q ∧ P := by
  intro pq
  cases pq with
  | intro p q =>
    constructor
    exact q
    exact p

```

After `intro pq` we are in the following state:

```

P Q : Prop
pq : P ∧ Q
⊢ Q ∧ P

```

Assuming $P \wedge Q$ is the same as assuming both P and Q . This is facilitated via the `cases` tactic which matches how a conjunction was created (and we learn the **constructor** for $\wedge$ is called `intro`). Hence after

```

cases pq with
  | intro p q =>

```

the state is

```

P Q : Prop

```

```

p : P
q : Q
⊢ Q ∧ P

```

The name `cases` seems to be a bit misleading since there is only one case to consider here. However, as we will see, `cases` is applicable more generally in situations where the name is better justified. And yes this is just pattern matching as you may have seen already in Haskell.

I hope you notice the same symmetry between tactics for **how to prove** and **how to use** which we have seen for implication also shows for conjunction. This pattern is going to continue.

It is good to know that Lean always abstracts the propositional variables we have declared. We can actually use `comAnd` with different instantiation to prove the following:

```

theorem comAndIff : P ∧ Q ↔ Q ∧ P :=
  constructor
  apply comAnd
  apply comAnd

```

Why is the first step `constructor`? Because as we said before

We define $P \leftrightarrow Q$ as $(P \rightarrow Q) \wedge (Q \rightarrow P)$.

(actually this is only approximately true but good enough at our current stage.)

In the second use of `comAnd` we instantiate `Q` with `P` and `P` with `Q`. Lean can usually infer these instantiations automatically from the goal, but in more complicated situations it may need a hint. We can write:

```

apply (comAnd (P:=Q) (Q:=P))

```

2.6. The Currying Equivalence

Maybe you have already noticed that a statement like $P \rightarrow Q \rightarrow R$ basically means that R can be proved from assuming both P and Q . Indeed, it is equivalent to $P \wedge Q \rightarrow R$. We can show this formally by using $\leftrightarrow$.

All the steps we have already explained, so we won't comment further here. It is a good idea to step through the proof (by hovering over the bubbles) or by using Lean.

```
theorem curry :  $P \wedge Q \rightarrow R \leftrightarrow P \rightarrow Q \rightarrow R$ 
constructor
• intro pqr p q
  apply pqr
constructor
• exact p
• exact q
• intro pqr pq
  cases pq with
  | intro p q =>
    apply pqr
    • exact p
    • exact q
```

I call this the currying equivalence, because this is the logical counterpart of currying in functional programming: that a function with several parameters can be reduced to a function which returns a function. For example, in Haskell addition has the type `Int -> Int -> Int` instead of `(Int, Int) -> Int`.

2.7. Disjunction

To prove a disjunction $P \vee Q$ we can either prove P or we can prove Q . This is achieved via the appropriately named tactics `left` and `right`.

Here are some simple examples:

```
example :  $P \rightarrow P \vee Q$  := by
  intro p
  left
  exact p
```

```
example :  $Q \rightarrow P \vee Q$  := by
  intro q
  right
  exact q
```

To use a disjunction $P \vee Q$ we have to show that the current goal follows both from assuming P and from assuming Q . To achieve this we use cases again, and this time the name actually makes sense. Here is an example:

```
theorem caseLem :  $(P \rightarrow R) \rightarrow (Q \rightarrow R)$  -
  intro p2r q2r pq
  cases pq with
  | inl p =>
    apply p2r
    exact p
  | inr q =>
    apply q2r
    exact q
```

After intro we are in the following state:

```
P Q R : Prop
p2r : P → R
q2r : Q → R
pq : P ∨ Q
⊢ R
```

We use

```
cases pq with
| inl p => ...
| inr q => ...
```

which means that we are going to use the assumption $P \vee Q$. There are two cases to consider, one for P and one for Q , resulting in two subgoals. We indicate which names we want to use for the assumptions in each case, namely p

and q :

```
2 goals
case Or.inl
P Q R : Prop
p2r : P → R
q2r : Q → R
p : P
⊢ R
```

```
case Or.inr
P Q R : Prop
p2r : P → R
q2r : Q → R
q : Q
⊢ R
```

To summarise: there are two ways to prove a disjunction using `left` and `right`. To **use** a disjunction in an assumption we use cases to perform a case analysis and show that the current goal follows from either of the two components.

2.8. Logic and algebra

As an example which involves both conjunction and disjunction we prove distributivity. In algebra we know the law $x(y+z) = xy + xz$; a similar law holds in propositional logic:

example : $P \wedge (Q \vee R) \leftrightarrow (P \wedge Q) \vee (P \wedge R)$
sorry $\square$

Sorry, but I was too lazy to do the proof so I left it to you. In Lean you can say `sorry` and omit the proof. This is an easy way out if you cannot complete your Lean homework: you just type `sorry` and it is fine. However, then I will say **Sorry but you don't get any points for this.**

The correspondence with algebra goes further: the counterpart to implication is exponentiation but you have to read it backwards, that is $P \rightarrow Q$

becomes $Q \wedge P$. Then the translation of the law $x^{yz} = (x^y)^z$ corresponds to the currying equivalence $P \wedge Q \rightarrow R \leftrightarrow P \rightarrow Q \rightarrow R$.

Maybe you remember that there is another law of exponentiation $x^{y+z} = x^y x^z$. And indeed its translation is also a law of logic:

```
theorem caseThm : P ∨ Q → R ↔ (P → R)
  sorry
```

2.9. True, false and negation

There are two logical constants `True` and `False`. It is a bit awkward to translate them into everyday English, so we think of `True` as something obviously true (e.g. **It sometimes rains in England**) and `False` as something obviously false (e.g. **Pigs can fly**).

As far as logic and proof are concerned, `True` behaves like an empty conjunction and `False` like an empty disjunction. Hence it is easy to prove `True`:

```
example : True := by
  constructor
```

While in the case of $\wedge$ we were left with two subgoals, here we are left with none—we are already done.

Symmetrically, there is no way to prove `False` (there is neither `left` nor `right`, and that's good!). On the other hand, doing cases on `False` as an assumption makes the current goal go away “by magic” and leaves no goals to be proven:

```
theorem efq : False → P := by
  intro pigs_can_fly
```

```
cases pigs_can_fly
```

`efq` is short for the Latin phrase **Ex falso**

quodlibet — “from falsehood, anything follows.”

This can feel odd in everyday reasoning, but in formal logic the inference “If pigs can fly then I am the president of America” is valid.

We define $\neg P$ as $P \rightarrow \text{False}$, which means that P is impossible. If someone says **If we get married then pigs can fly**, that means **no**.

As an example, we can prove the law of contradiction: it cannot be that both P and $\neg P$ hold.

```
theorem contr :  $\neg (P \wedge \neg P)$  := by
  intro pnp
  cases pnp with
  | intro p np =>
    apply np
    exact p
```

2.10. Use have

There is one more useful tactic which is not related to any propositional connective. Let's say we want to prove

$$(P \rightarrow Q \vee Q) \rightarrow (P \rightarrow Q)$$

Now it is pretty easy to **cut** this proof by showing that $Q \vee Q \rightarrow Q$ and then use this to show the result. But there is a better way which avoids introducing a new theorem and which is called **have**. This is best explained by an example:

```
example :  $(P \rightarrow Q \vee Q) \rightarrow (P \rightarrow Q)$  := by
  intro h p
  have qorq :  $Q \vee Q$  := by
    apply h
    exact p
  cases qorq with
  | inl q =>
```

```

      exact q
    | inr q =>
      exact q

```

We introduce a new goal by saying

```
have qorq : Q ∨ Q := by
```

and now the proof state is

```

h : P → Q ∨ Q
p : P
⊢ Q ∨ Q

```

In the next lines we prove this and afterwards we can use `qorq`

```

h : P → Q ∨ Q
p : P
qorq : Q ∨ Q

```

So we use `have` to introduce intermediate goals. This is called a **cut**.

2.11. Summary of tactics

Below is a table summarising the tactics we have seen so far (Lean 4 syntax):

	How to prove?	How to use?
$\rightarrow$	<code>intro h</code>	<code>apply h</code>
$\wedge$	<code>constructor</code>	<code>cases h with </code> <code>intro p q => ...</code>
$\vee$	<code>left / right</code>	<code>cases h with </code> <code>inl p => ... </code> <code>inr q => ...</code>
<code>True</code>	<code>constructor</code>	<code>—</code>

False — cases h

These correspond to the introduction and elimination rules in **natural deduction**, a system devised by the German logician Gerhard Gentzen.



Gerhard Gentzen

(1909–1945)

He also proved that in his system cuts can always be avoided (called the **cut elimination theorem** or the **Hauptsatz**).

The surface syntax for using conjunction and disjunction looks similar—both use cases—but the effect is different. For $\wedge$, both components become available in the single subgoal; for $\vee$, you get two subgoals, one per alternative.

We also have $\text{exact } h$, which is a structural tactic that doesn't fit the scheme above (and more generally $\text{exact } t$ for any proof term t). Another structural tactic is have for introducing intermediate goals. There is also assumption , which checks whether any assumption matches the current goal. Thus we could have written the first proof as:

example : $P \rightarrow P$:= **by** $\square$

intro h $\square$
assumption $\square$

Important. There are many more tactics available in Lean, some with a higher degree of automation, and some tactics can be used in ways we have not discussed here. When solving exercises, please use only the tactics we have introduced and only in the ways we have described.

← 1. Introduction

3. Classical Logic →

IFR (COMP2065) lecture notes

Table of Contents

1. Introduction
2. Propositional Logic
3. Classical Logic
4. Predicate Logic
5. The Booleans
6. The Natural Numbers
7. Lists
8. Trees

3. Classical Logic

- 3.1. The de Morgan laws
- 3.2. The law of the excluded middle
- 3.3. Indirect proof
- 3.4. Intuitionistic vs classical logic

← 2. Propositional Logic

4. Predicate Logic →

3. Classical Logic

We stick to propositional logic for the moment but discuss a difference between the logic based on truth you may have seen before and the logic based on evidence which we have introduced in the previous chapter.

The truth-based logic is called **classical logic** while the evidence-based one is called **intuitionistic logic**.

3.1. The de Morgan laws

The de Morgan laws state that if you negate a disjunction or conjunction this is equivalent to the negation of their components with the disjunction replaced by conjunction and vice versa. More precisely:

$$\neg (P \vee Q) \leftrightarrow \neg P \wedge \neg Q$$

$$\neg (P \wedge Q) \leftrightarrow \neg P \vee \neg Q$$

These laws reflect the observation that the truth tables for $\wedge$ and $\vee$ can be transformed into each other if we turn them around and swap True and False.

P	Q	P $\wedge$ Q	P $\vee$ Q	$\neg P$	$\neg Q$	$\neg(P \wedge Q)$	$\neg(P \vee Q)$
False	False	False	False	True	True	True	True
True	False	False	True	False	True	True	False
False	True	False	True	True	False	True	False
True	True	True	True	False	False	False	False

Here is the proof of the first de Morgan law in its full glory.

```

theorem dm1 :  $\neg (P \vee Q) \leftrightarrow \neg P \wedge \neg Q := \text{by}$ 
  constructor
  • intro npq
    constructor
    • intro p
      apply npq
      left
      exact p
    • intro q
      apply npq
      right
      exact q
  • intro npnq pq
    cases npnq with
    | intro np nq =>
      cases pq with
      | inl p =>
        apply np
        exact p
      | inr q =>
        apply nq
        exact q

```

It is rather boring because there are a lot of symmetric cases but I didn't break a sweat proving it. However, the 2nd law is a different beast. Here is my attempt (left-to-right direction gets stuck):

```

theorem dm2_attempt :  $\neg (P \wedge Q) \leftrightarrow \neg P \vee \neg Q :$ 
  constructor
  • intro npq
    left
    intro p
    apply npq
    constructor
    exact p
    -- stuck here
    sorry
  • intro npnq pq
    cases pq with
    | intro p q =>
      cases npnq with
      | inl np =>
        apply np
        exact p
      | inr nq =>
        apply nq
        exact q

```

As you see I got stuck with the left to right direction, the right to left one went fine. What is the problem? The proof state after `intro npq` is (ignoring the propositional assumptions and the other goal):

$$\begin{array}{l} \text{npq} : \neg(P \wedge Q) \\ \vdash \neg P \vee \neg Q \end{array}$$

Now the question is do we go left or right - there seems to be no good reason for either because everything is symmetric. Ok let's try left we end up with:

$$\begin{array}{l} \text{npq} : \neg(P \wedge Q) \\ \vdash \neg P \end{array}$$

Now the next steps is obvious intro p:

$$\begin{array}{l} \text{npq} : \neg(P \wedge Q), \\ p : P \\ \vdash \text{false} \end{array}$$

There is only one purveyor of False, hence we say apply npq:

$$\begin{array}{l} \text{npq} : \neg(P \wedge Q), \\ p : P \\ \vdash P \wedge Q \end{array}$$

Now we say constructor and the first subgoal is easily disposed with exact p but we end up with:

$$\begin{array}{l} \text{npq} : \neg(P \wedge Q), \\ p : P \\ \vdash Q \end{array}$$

And there is no good way to make progress here, indeed it could be that P is true but Q is false. As soon as we said left we ended up with an unprovable goal.

What has happened? The truth tables provided clear evidence that the de Morgan law should hold but we couldn't prove it. Indeed let's consider the following example: **It is not the case that I have a cat and that I have a dog** can we conclude that **I don't have a cat or I don't have a dog**? No because we don't know which one is the case, that is we don't have evidence for either.

3.2. The law of the excluded middle

To match the truth semantics we need to assume one axiom, the **law of the excluded middle** (em). This expresses the idea that every proposition is either true or false: $P \vee \neg P$

for any proposition P .

```
open Classical
```

```
#check em
```

```
| Classical.em (p : Prop) : p ∨ ¬p
```

Using `em P`, we can prove the missing direction of the 2nd de Morgan law (and hence the equivalence $\neg (P \wedge Q) \leftrightarrow \neg P \vee \neg Q$).

```
theorem dm2_em : ¬ (P ∧ Q) → ¬ P ∨ ¬ Q := by
  intro npq
  have pnp : P ∨ ¬ P := by
    apply em
  cases pnp with
  | inl p =>
    right
    intro q
    apply npq
    constructor
    · exact p
    · exact q
  | inr np =>
    left
    exact np
```

The idea of the proof is that we look at both cases of $P \vee \neg P$. If P holds then we can prove $\neg Q$ from $\neg (P \wedge Q)$, otherwise if we know $\neg P$ then we can obviously prove $\neg P \vee \neg Q$.

3.3. Indirect proof

There is another law which is equivalent to the principle of excluded middle and this is the **principle of indirect proof** or in latin **reduction ad absurdo** (reduction to the absurd). This principle tells you that to prove P it is sufficient to show that $\neg P$ is impossible. Here is how we derive this using `em`:

```
theorem byContradiction : ¬¬ P → P := by
  intro nnp
  have pnp : P ∨ ¬ P := by
    apply em
  cases pnp with
  | inl p =>
    exact p
  | inr np =>
    have pcf : False := by
      apply nnp
```

```

exact np
cases pcf

```

The idea is to assume $\neg \neg P$ and then prove P by analysing $P \vee \neg P$: In the case P we are done and in the case $\neg P$ we have a contradiction with $\neg \neg P$ and we can use that false implies everything.

We can also derive em from byContradiction . The key observation is that we can actually prove $\neg \neg (P \vee \neg P)$ without using classical logic.

```

theorem nnEm :  $\neg \neg (P \vee \neg P)$  := by
  intro npnp
  apply npnp
  right
  intro p
  apply npnp
  left
  exact p

```

This proof is a bit weird. After `apply npnp` we have the following state:

```

P : Prop,
npnp :  $\neg(P \vee \neg P)$ 
⊢  $P \vee \neg P$ 

```

Now you may say again do we go left or right? But this time the cases are not symmetric. we certainly cannot prove P hence let's go right. After a few steps we are in the same situation again:

```

P : Prop,
npnp :  $\neg(P \vee \neg P)$ ,
p : P
⊢  $P \vee \neg P$ 

```

But something has changed! We have picked up the assumption $p : P$. And hence this time we go left and we are done.

Here is a little story which relies on the idea that double negating corresponds to time travel:

"There was a magician who could time travel who wanted to marry the daughter of a king. There was no gold in the country but people were not sure whether diamonds exist. Hence the king set the magician the task to either find a diamond or to produce a way to turn diamonds into gold.

The magician went for the 2nd option and gave the king an empty box so he could marry the daughter. However, if the king would get hold of a diamond at some point and his lie would become obvious he would just take the diamond, travel back in time and go for the first option."

Now if we assume we have an axiom proving `byContradiction` we can show `em`:

```
axiom AxbyContradiction :  $\neg \neg P \rightarrow P$ 
```

```
theorem em :  $P \vee \neg P := \text{by}$   
  apply AxbyContradiction  
  apply nnEm
```

Note that while `em` and `raa` are equivalent as global principles this is not the case for individual propositions. That is if we assume $P \vee \neg P$ we can prove $\neg \neg P \rightarrow P$ for the same proposition P but if we assume $\neg \neg P \rightarrow P$ we cannot prove $P \vee \neg P$ for that proposition but we actually need a different instance of `raa` namely : $\neg \neg (P \vee \neg P) \rightarrow P \vee \neg P$.

3.4. Intuitionistic vs classical logic

Should we always assume `em` (or alternatively `raa`), hence should we always work in classical logic? There is a philosophical and a pragmatic argument in favour of avoiding it and using intuitionistic logic.

The philosophical argument goes like this: while facts about the real world are true or false even if we don't know them this isn't so obvious about mathematical constructions which take place in our head. The set of all numbers doesn't exist in the real world it is like a story we share and we don't know whether anything we make up is either true or false. However, we can talk about evidence without needing to assume that.

The idea that the world of ideas is somehow real, and that the real world is just a poor shadow of the world of ideas was introduced by the greek philosopher Plato and hence is called **Platonism**. In contrast that our ideas are just constructions in our head is called **Intuitionism**.

However, the pragmatic argument is maybe more important. Intuitionistic logic is constructive, indeed in a way that is dear to us computer scientists: whenever we show that something exists we are actually able to compute it. As a consequence intuitionistic logic introduces many distinctions which are important especially in computer science. For example we can distinguish decidable properties from properties in general. Also by a function we mean something we can compute like in a (functional) programming language.

Here is a famous example to show that the principle of excluded middle destroys constructivity. Since we haven't yet introduced predicate logic and many of the concepts needed for this example in Lean I present this just as an informal argument:

We want to show that there are two irrational numbers p and q (that is numbers that cannot be written as fractions) such that their power p^q is rational. We know that $\sqrt{2}$ is irrational. Now what is $\sqrt{2}^{\sqrt{2}}$? Using the excluded middle it is either rational or irrational. If it is rational then we are done $p = q = \sqrt{2}$. Otherwise we use $p = \sqrt{2}^{\sqrt{2}}$ which we now assume to be irrational and $q = \sqrt{2}$. Now a simple calculation shows $p^q = (\sqrt{2}^{\sqrt{2}})^{\sqrt{2}} = \sqrt{2}^{\sqrt{2}\sqrt{2}} = \sqrt{2}^2 = 2$ which is certainly rational.

Now after this proof we still don't know two irrational numbers whose power is rational. I hasten to add that we can establish this fact without using em but this particular proof doesn't provide a witness because it is using excluded middle.

In the homework we will distinguish proofs that use excluded middle and once which do not. In my logic poker I ask you to prove propositions intuitionistically where possible and only use classical reasoning where necessary.

Now a common question is how to see whether a proposition is only provable classically. E.g. why can we prove all the first de Morgan law and one direction of the 2nd but not $\neg (P \wedge Q) \rightarrow \neg P \vee \neg Q$? The reason is that the right hand side contains some information, i.e. which of the $\neg P$ or $\neg Q$ is true while $\neg (P \wedge Q)$ is a **negative** proposition and hence

does contain no information. In contrast the both sides of the first de Morgan law $\neg (P \vee Q)$ and $\neg P \wedge \neg Q$ are negative, i.e. contain no information.

← 2. Propositional Logic

4. Predicate Logic →

IFR (COMP2065) lecture Search...

Table of Contents

1. Introduction
2. Propositional Logic
3. Classical Logic
4. Predicate Logic
5. The Booleans
6. The Natural Numbers
7. Lists
8. Trees

4. Predicate Logic

- 4.1. Predicates, relations and quantifiers
- 4.2. Quantifiers
- 4.3. The universal quantifier
- 4.4. The existential quantifier
- 4.5. Another Currying equivalence
- 4.6. Equality
- 4.7. Classical Predicate Logic
- 4.8. Summary of tactics

← 3. Classical Logic

5. The Booleans →

4. Predicate Logic

4.1. Predicates, relations and quantifiers

Predicate logic extends propositional logic, we can use it to talk about objects and their properties. The objects are organized in **types**, such as $\mathbb{N}$: Type the type of natural numbers $\{0, 1, 2, 3 \dots\}$ or `Bool` : Type the type of Booleans $\{\text{true}, \text{false}\}$, or lists over a given A : Type: `list A` : Type, which we will introduce in more detail soon.

To avoid talking about specific types which we will introduce later we introduce some type variables:

```
variable {A B C : Type}
```

We talk about types where you may be used to **sets**. While they are subtle differences (types are static while we can reason about set membership in set theory) for our purposes types are just a replacement of sets.

We also have function types $A \rightarrow B$ which you know from functional programming. So for example we can define

```
def double :  $\mathbb{N} \rightarrow \mathbb{N}$ 
| n => n + n

def foo :  $\mathbb{N} \rightarrow \mathbb{N} \rightarrow \mathbb{N}$ 
| m , n => m + n + n
```

As in Haskell functions with several arguments are curried but note that you have to separate the parameters by `,`. An alternative is to use explicit function definitions:

```
def double' (n :  $\mathbb{N}$ ) :  $\mathbb{N}$ 
:= n + n

def foo' (m n :  $\mathbb{N}$ ) :  $\mathbb{N}$ 
:= m + n + n
```

As in Haskell function application is written without brackets, e.g.

```
#eval double 2
#eval foo' 2 3
```

A predicate is just another word for a property, e.g. we may use `Prime :  $\mathbb{N} \rightarrow \text{Prop}$` to express that a number is a prime number. Note that a predicate is just a function!

We can form propositions such as `Prime 3` and `Prime 4`, the first one should be provable while the negation of the second holds. Predicates may have several inputs in which case we usually call them relations, examples are `Nat.le :  $\mathbb{N} \rightarrow \mathbb{N} \rightarrow \text{Prop}$` or `List.mem : A → list A → Prop` to form propositions like `Nat.le 2 3` and `List.mem 1 [1,2,3]` (both of them should be provable). Using defined notations these two examples can also be written as `2 ≤ 3` and `1 ∈ [1,2,3]`.

In the sequel we will use some generic predicates for examples, such as

```
variable {PP QQ : A → Prop}
```

4.2. Quantifiers

The most important innovation of predicate logic

are the quantifiers, which we can use to form new propositions:

- universal quantification ($\forall$), read $\forall x : A$, $PP\ x$ as all x in A satisfy $PP\ x$.
- existential quantification ($\exists$), read $\exists x : A$, $PP\ x$ as there is an x in A satisfying $PP\ x$.

Here are some examples of propositions in predicate logic:

#check $\forall x : A, PP\ x \wedge Q$
#check $(\forall x : A, PP\ x) \wedge Q$
#check $\forall x:A, (\exists x : A, PP\ x) \wedge QQ$
#check $\forall y:A, (\exists z : A, PP\ z) \wedge QQ$

Both quantifiers bind weaker than any other propositional operator, that is we read $\forall x : A, PP\ x \wedge Q$ as $\forall x : A, (PP\ x \wedge Q)$. We need parentheses to limit the scope, e.g. $(\forall x : A, PP\ x) \wedge Q$ which has a different meaning to the proposition before.

It is important to understand bound variables, essentially they work like scoped variables in programming. We can shadow variables as in $\forall x:A, (\exists x : A, PP\ x) \wedge QQ\ x$, here the x in $PP\ x$ refers to $\exists x : A$ while the x in $QQ\ x$ refers to $\forall x : A$. Bound variables can be consistently renamed, hence the previous proposition is the same as $\forall y:A, (\exists z : A, PP\ z) \wedge QQ\ y$, which is actually preferable since shadowing variables should be avoided because it confuses the human reader.

Now we have introduced all these variables what can we do with them. We have a new primitive proposition:

- equality ($=$), given $a\ b : A$ we write $a = b$ which we read as a is equal to b .

In the moment we only have variables as elements of types but this will change soon when we introduce datatypes and functions.

4.3. The universal quantifier

To prove that a proposition of the form $\forall x : A, PP\ x$ holds we assume that there is given an arbitrary element a in A and prove it for this generic element, i.e. to prove $PP\ a$, we use `intro a` to do this.

If we have an assumption $h : \forall x : A, PP\ x$ and our current goal is $PP\ a$ for some $a : A$ then we can use `apply h` to prove our goal. Usually we have some combination of implication and forall like $h : \forall x : A, PP\ x \rightarrow QQ\ x$ and now if our current goal is $QQ\ a$ and we invoke `apply h` Lean will instantiate x with a and it remains to show $PP\ a$.

Best to do some examples. Let's say we want to prove

$$(\forall x : A, PP\ x) \rightarrow (\forall y : A, PP\ y \rightarrow QQ\ y) \rightarrow \forall z : A, QQ\ z$$

Here is a possible translation into English where we assume that A stands for the type of students in the class, $PP\ x$ means **x is clever** and $QQ\ x$ means **x is funny** then we arrive at:

If all students are clever then if all clever students are funny then all students are funny.

```
example : (∀ x : A, PP x)
  → (∀ y : A, PP y → QQ y)
  → ∀ z : A, QQ z := by
  intro p pq a
  apply pq
  apply p
```

Note that after `intro` the proof state is:

$$p : \forall (x : A), PP\ x, pq : \forall (y : A), PP\ y \rightarrow QQ\ y, a : A \\ \vdash QQ\ a$$

That is the `x` in `QQ x` has been replaced by `a`. I could have used `x` again but I thought this may be misleading because you may think that you have to use the same variable as in the quantifier.

Let's prove a logical equivalence involving $\forall$ and $\wedge$, namely that we can interchange them. That is we are going to prove

$$(\forall x : A, PP\ x \wedge QQ\ x) \leftrightarrow (\forall x : A, PP\ x) \wedge (\forall x : A, QQ\ x)$$

To illustrate this: to say that **all students are clever and funny** is the same as saying that **all students are clever and all students are funny**.

Here is the Lean proof:

```
example : (∀ x : A, PP x ∧ QQ x)
  ↔ (∀ x : A, PP x) ∧ (∀ x : A, QQ x) :=
  constructor
  intro h
  constructor
  intro a
  have pq : PP a ∧ QQ a := by
    apply h
  cases pq with
  | intro pa qa => exact pa
  intro a
  have pq : PP a ∧ QQ a := by
    apply h
  cases pq with
  | intro pa qa => exact qa
  intro h
  cases h with
  | intro hp hq =>
    intro a
    constructor
    apply hp
    apply hq
```

I am using `have` which we have already seen in the chapter on Propositional Logic. After `intro a` I am in the following state (ignoring the parts not relevant now):

```
h : ∀ (x : A), PP x ∧ QQ x,
a : A
⊢ PP a
```

Now I cannot say `apply h` because `PP a` is not the conclusion of the assumption. My idea is that I can prove `PP a ∧ QQ a` from `h` and from this I can prove `PP a`. Hence I am using

```
have pq : PP a ∧ QQ a := by
  apply h
```

and then I can proof `PP a` by using `cases` on `pq`.

4.4. The existential quantifier

To prove a proposition of the form $\exists x : A, PP\ x$ it is enough to prove `PP a` for some $a : A$. We say `exists a` and are left to prove `PP a`.

On the other hand to use an assumption of the form $h : \exists x : A, P\ x$ we are using

```
cases h with
| intro x px => ...
```

which replaces `h` with two assumptions $x : A$ and $px : P\ x$.

Again it is best to look at an example. We are going to prove a proposition very similar to the one for $\forall$:

$$(\exists x : A, PP\ x) \rightarrow (\forall y : A, PP\ y \rightarrow QQ\ y) \rightarrow \exists z : A, QQ\ z$$

Here is the english version using the same translation as before:

If there is a clever student and all clever students are funny then there is a funny student.

Here is the Lean proof (Lean 4):

```
example :
  (∃ x : A, PP x) →
  (∀ y : A, PP y → QQ y) →
  ∃ z : A, QQ z := by
  intro p pq
  cases p with
  | intro a pa =>
    exists a
    apply pq
    apply pa
```

After the `intro` we are in the following state:

```
p : ∃ (x : A), PP x,
pq : ∀ (y : A), PP y → QQ y
⊢ ∃ (z : A), QQ z
```

We first take `p` apart using `cases`:

```
pq : ∀ (y : A), PP y → QQ y,
a : A,
pa : PP a
⊢ ∃ (z : A), QQ z
```

and now we can use `exists a`. We are left with

```
pq : ∀ (y : A), PP y → QQ y
a : A
pa : PP a
⊢ QQ a
```

which is easy to prove.

As $\forall$ can be exchanged with $\wedge$, $\exists$ can be exchanged with $\vee$. That is we are going to prove the following equivalence:

$$(\exists x : A, PP x \vee QQ x) \leftrightarrow (\exists x : A,$$

$$PP\ x) \vee (\exists\ x : A, QQ\ x)$$

Here is the english version

There is a student who is clever or funny is the same as saying there is a student who is funny or there is a student who is clever.

Here is the complete Lean proof (for you to step through online):

```
example :
  (∃ x : A, PP x ∨ QQ x) ↔
  (∃ x : A , PP x) ∨ (∃ x : A, QQ x) :
  constructor
  • intro h
    cases h with
    | intro a ha =>
      cases ha with
      | inl pa =>
        left
        exists a
        -- exact pa, not needed
      | inr qa =>
        right
        exists a
        -- exact qa, not needed
  • intro h
    cases h with
    | inl hp =>
      cases hp with
      | intro a pa =>
        exists a
        left
        exact pa
    | inr hq =>
      cases hq with
      | intro a qa =>
        exists a
        right
        exact qa
```

4.4.1. Use constructor for $\exists$

There is an alternative to `exists` to prove $\exists$ which shows the similarity of $\exists$ and $\wedge$: we can use `constructor` to prove `exists`. Let's revisit our

first example

```

example :
  ( $\exists x : A, PP\ x$ )  $\rightarrow$ 
  ( $\forall y : A, PP\ y \rightarrow QQ\ y$ )  $\rightarrow$ 
   $\exists z : A, QQ\ z := \mathbf{by}$ 
  intro  $p\ pq$ 
  cases  $p$  with
  | intro  $a\ pa \Rightarrow$ 
    constructor
    apply  $pq$ 
    assumption

```

After constructor we are in the following state (ignoring the assumptions introduced by variable):

```

case intro.h
pq :  $\forall (y : A), PP\ y \rightarrow QQ\ y$ 
a : A
pa : PP a
 $\vdash QQ\ ?intro.w$ 
case intro.w
pq :  $\forall (y : A), PP\ y \rightarrow QQ\ y$ 
a : A
pa : PP a
 $\vdash A$ 

```

Indeed we have two goals: the 2nd one is finding the witness, an element of A, which is displayed as $\vdash A$ and the first one is to show that QQ holds for that witness which is $QQ\ ?intro.w$ where $?intro.w$ refers to the other yet incomplete goal. Instead of finding a witness first (as with exists) we just try to prove $QQ\ ?intro.w$ and hope that the other goal gets instantiated automatically. Which is what happens because after **apply** pq we see

```

pq :  $\forall (y : A), PP\ y \rightarrow QQ\ y$ 
a : A
pa : PP a
 $\vdash PP\ ?intro.w$ 

```

and once we prove this using **assumption** lean will use pa and hence knows that $?intro.w$ must

be a . So this goal gets discharged automatically.

This is clearly better because it saves us the need to construct the witness manually. However, some care is in place when using `constructor`. We may try to postpone the cases until the end

```
example (A : Type) (PP QQ : A → Prop)
  (∃ x : A, PP x) →
  (∀ y : A, PP y → QQ y) →
  ∃ z : A, QQ z := by
  intro p pq
  constructor
  apply pq
  cases p with
  | intro a pa =>
    assumption
```

Before the last step we are in the following state

```
pq : ∀ (y : A), PP y → QQ y
a : A
pa : PP a
⊢ PP (?m.27 ...)
```

but `assumption` fails because the weird looking `PP (?m.27 ...)` cannot be instantiated with a . The problem has to do with scope we have introduced the goal $\vdash A$ **before** we introduced a when doing `cases`. Indeed the situation is similar as with $\wedge$ where we would replicate code if we do `cases` later. However, here doing things in the wrong order means it is impossible to complete the proof.

4.5. Another Currying equivalence

You may have noticed that the way we prove propositions involving $\rightarrow$ and $\forall$ is very similar. In both cases we use `intro` to prove them by introducing an assumption in the first case a

proposition and in the second an element in a type and in both cases we use them using `apply` to prove the current goal. Similarly $\wedge$ and $\exists$ behave similarly: we need to supply two components. And in both cases we are using cases with two components which basically replaces the assumption by its two components.

The similarity can be seen by establishing another currying-style equivalence. While currying in propositional logic had the form

$$P \wedge Q \rightarrow R \leftrightarrow P \rightarrow Q \rightarrow R$$

where we turn a conjunction into an implication, currying for predicate logic has the form

$$(\exists x : A, Q x) \rightarrow R \leftrightarrow (\forall x : A, Q x \rightarrow R)$$

this time we turn an existential into a universal quantifier. For the intuition, we use $Q x$ to mean **x is clever** and R means **the professor is happy**. Hence the equivalence is:

If there is a student who is clever then the professor is happy is equivalent to saying if any student is clever then the professor is happy.

Here is the proof in Lean:

```

theorem curryPred :
  (( $\exists x : A, P x$ )  $\rightarrow R$ )  $\leftrightarrow$  ( $\forall x :$ 
  constructor
  • intro ppr
    intro a p
    apply ppr
    exists a
  • intro ppr
    intro pp
    cases pp with
    | intro a p =>
      exact ppr a p

```

4.6. Equality

There is a generic relation which can be applied to any type: **equality**. Given $a, b : A$ we can construct $a = b : \text{Prop}$ expressing that a and b are equal. We can prove that everything is equal to itself using the tactic `rfl` (or reflexivity).

```
example :  $\forall x : A, x = x := \text{by}$   
  intro a  
  rfl
```

If we have assumed an equality $h : a = b$ we can use it to **rewrite** a into b in the goal. That is if our goal is $\text{PP } a$ we say `rw [h]` and this changes the goal into $\text{PP } b$. Here is a simple example (with a little twist):

```
example :  $\forall x y : A, x = y \rightarrow \text{PP } y \rightarrow 1$   
  intro x y  $\text{eq}$   $p$   
  rw [ $\text{eq}$ ]  
  exact  $p$ 
```

Sometimes we want to use the equality in the other direction, that is we want to replace b by a . In this case we use `rw [← h]`. Here is another example which is actually what I wanted to do first:

```
example :  $\forall x y : A, x = y \rightarrow \text{PP } x \rightarrow 1$   
  intro x y  $\text{eq}$   $p$   
  rw [←  $\text{eq}$ ]  
  exact  $p$ 
```

Equality is an **equivalence relation**, it means that it is

- reflexive ($\forall x : A, x = x$),
- symmetric ($\forall x y : A, x = y \rightarrow y = x$)
- transitive ($\forall x y z : A, x = y \rightarrow y = z \rightarrow x = z$)

We have already shown reflexivity using `rf1`. We can show symmetry and transitivity using `rw`:

```
theorem symEq :  $\forall x y : A, x = y \rightarrow y = x$ 
  intro x y  $\square$  p  $\square$ 
  rw [p]  $\square$ 
```

After the `intro` the goal is:

```
x y : A,
p : x = y
⊢ y = x
```

After `rw [p]` the goal becomes `y = y` which is solved automatically because `rw` applies reflexivity if possible.

Moving on to transitivity:

```
theorem transEq :  $\forall x y z : A, x = y \rightarrow y = z \rightarrow x = z$ 
  intro x y  $\square$  z  $\square$  xy  $\square$  yz  $\square$ 
  rw [xy]  $\square$ 
  exact yz  $\square$ 
```

Sometimes we want to use an equality not to rewrite the goal but to rewrite another assumption. We can do this using `at`, giving another proof of transitivity:

```
theorem transEq' :  $\forall x y z : A, x = y \rightarrow y = z \rightarrow x = z$ 
  intro x y  $\square$  z  $\square$  xy  $\square$  yz  $\square$ 
  rw [← xy] at yz  $\square$ 
  exact yz  $\square$ 
```

Lean also provides built-in tactics to deal with symmetry and transitivity:

```
example :  $\forall x y : A, x = y \rightarrow y = x$  :=
  intro x y  $\square$  p  $\square$ 
  symm  $\square$ 
  exact p  $\square$ 

example :  $\forall x y z : A, x = y \rightarrow y = z \rightarrow x = z$ 
  intro x y  $\square$  z  $\square$  xy  $\square$  yz  $\square$ 
  apply transEq  $\square$ 
  exact xy  $\square$ 
  exact yz  $\square$ 
```

After we say `trans` the goals are split:

```
2 goals
⊢ x = ?m_1
⊢ ?m_1 = z
```

Lean will infer `?m_1 := y` after we solve the first subgoal with `exact xy`.

A very nice way to do equational proofs is to use `calc`:

```
example : ∀ x y z : A, x = y → y = z
intro x y○ z○ xy○ yz○
calc
  x = y := by○ exact xy○
  _ = z := by○ exact yz○
```

Here each line shows an equality and its justification; `_` refers to the right side of the previous line.

Finally, equality is a **congruence**: functions preserve equality. If `f : A → B` and `x = y`, then `f x = f y`:

```
example : ∀ f : A → B, ∀ x y : A, x =
intro f x○ y○ h○
rw [h]○
```

4.7. Classical Predicate Logic

We can use classical logic in predicate logic even though the explanation using truth tables does not scale well. Think of them as "infinite truth tables."

There are predicate-logic counterparts of de Morgan's laws, which say that you can move negation through a quantifier by negating the component and switching the quantifier.

4.7.1. First de Morgan law (intuitionistic proof)

```

theorem dm1_pred {A : Type} {PP : A →
  ¬ (∃ x : A, PP x) ↔ ∀ x : A, ¬ PP x
constructor
• intro h x px
  apply h
  constructor
  apply px
• intro h np
  cases np with
  | intro a pa =>
    apply h
    apply pa

```

Neither direction of this law requires classical logic.

4.7.2. Second de Morgan law (needs classical logic)

We need **reductio ad absurdum**
(byContradiction), which depends on
excluded middle. Now the de Morgan law:

```

theorem dm2Pred {A : Type} {PP : A →
  ¬ (∀ x : A, PP x) ↔ ∃ x : A, ¬ PP x
constructor
• intro h
  apply byContradiction
  intro ne
  apply h
  intro a
  apply byContradiction
  intro np
  apply ne
  constructor
  apply np
• intro h na
  cases h with
  | intro a np =>
    apply np
    apply na

```

We needed byContradiction twice in the first direction.

4.7.3. Intuition

The existential statement $\exists x : A, \neg P x$ contains explicit information (it points to a witness a), but the negated universal $\neg (\forall x : A, P x)$ does not. Intuitively: **knowing that not all students are stupid does not give you a way to name a student who is not stupid.**

4.8. Summary of tactics

Here is the summary of basic tactics for predicate logic:

How to prove?	How to use?
$\forall$ intro h	<code>apply h</code>
$\exists$ exists $a /$ constructor	<code>cases h with </code> <code>intro x p => ...</code>
<code>= rfl</code>	<code>rw [h] / rw [← h]</code>
← 3. Classical Logic	5. The Booleans →

IFR (COMP2065) lecture

Table of Contents

1. Introduction
2. Propositional Logic
3. Classical Logic
4. Predicate Logic
5. The Booleans
6. The Natural Numbers
7. Lists
8. Trees

5. The Booleans

- 5.1. Proving some basic properties
- 5.2. Functions on Bool
- 5.3. Proving equations about Bool
- 5.4. Relating Bool and Prop
- 5.5. No confusion from first principles

[← 4. Predicate Logic](#)[6. The Natural Numbers](#)[→](#)

5. The Booleans

The logic we have introduced so far was very generic. We fix this in this chapter by looking at a very simple type, the Booleans `Bool` which has just two elements `true` and `false` and functions on this type. Then we are going to use predicate logic to prove some simple theorems about Booleans.

`Bool` is defined as an inductive type (this is just a copy of the definition in the Lean prelude).

```
inductive Bool : Type
| false : Bool
| true  : Bool
```

This declaration means:

- There is a new type `Bool : Type`,
- There are two elements `true false : Bool`,
- These are the only elements of `Bool`,
- `true` and `false` are different elements of `Bool`.

The `inductive` keyword is quite versatile: we can use it to define other finite types, infinite types like $\mathbb{N}$, and type constructors like `Option` (Maybe in Haskell) or `List`. It is similar to the `data` type constructor in Haskell, but not exactly, since there are data definitions in Haskell which are not permitted in Lean.

5.1. Proving some basic properties

How do we prove facts about `Bool`? Let's prove the first basic property that `true` and `false` are the only elements of `Bool`:

```
theorem allBool :  $\forall$  b : Bool, b=true
intro b
cases b with
| true =>
  left
  rfl
| false =>
  right
  rfl
```

Here we use `cases` again. Our goal is

```
b : Bool
⊢ b = true ∨ b = false
```

using `cases` we reduce this to two goals :

```
case false
⊢ false = true ∨ false = false
case true
⊢ true = true ∨ true = false
```

both of them are easy to prove.

To show that `true` and `false` are different we need to do a case on equality using the fact that Lean already know that it is impossible to prove a equality between different constructors. The proof of this **no confusion theorem** looks like this:

```
theorem noConfBool : true ≠ false :=
intro q
cases q
```

We use the fact that `true ≠ false` is defined as $\neg (true = false)$ and hence `true = false` $\rightarrow$ `False`. Hence after `intro q` we are in the

following state:

```
q : true = false
⊢ False
```

Now there is no way to prove `False` but we can ask Lean to analyze our assumption `q` and it knows that this is impossible.

You may think that we are cheating here a bit because basically Lean already knows that there is no proof of `true = false`. Indeed we can actually prove `true ≠ false` from first principles using only `rewrite` and a function `Bool → Prop` (see next section for functions on `Bool`). And this is the way it is done in the Lean library.

5.2. Functions on Bool

Let's define negation on Booleans as a function. We define the function by matching all possible inputs:

```
def not : Bool → Bool
| true => false
| false => true
```

To define a function with two inputs, such as the boolean conjunction (we'll implement it as `and`), we use **currying**.

We could list all four cases, reproducing the full truth table, but we can get away with just two by matching only on the first argument:

```
def and : Bool → Bool → Bool
| true , b => b
| false , _ => false
```

If the first argument is `true`, i.e. we look at `and true : Bool → Bool`, then this is just the identity on the second argument, because `and`

`true true = true` and `and true false = false`. If the first argument is `false` then the outcome will be `false` whatever the second argument is. In other words `and false : Bool → Bool` is the constant function which always returns `false`.

Symmetrically we can define disjunction (implemented as `bor`) with just two cases:

```
def or : Bool → Bool → Bool
| true , _ => true
| false , b => b
```

In this case, if the first argument is `true`, then `or true : Bool → Bool` is always `true`, while if the first argument is `false`, `bor false : Bool → Bool` is just the identity.

We use the standard notation for boolean operators, i.e.

`! b` **`not b`**

`b && c` `b and c`

`b || c` `b or c`

Note that the precedence is the same as for the propositional operations which means that we always have to use brackets when writing equations involving boolean operators otherwise `x && y = z` is read as `x && (y = z)`. Hence we write `(x && y) = z`.

If you wonder why `x && (y = z)` even makes sense we will come to that later.

We can try this out by evaluating a boolean expression:

```
#eval ! false || false && true
| true
```

When we defined these binary boolean functions, our choice to match on the first argument was quite arbitrary — we could have matched on the second argument instead:

```
def and' : Bool → Bool → Bool
| b , true => b
| _ , false => false

def or' : Bool → Bool → Bool
| _ , true => true
| b , false => b
```

These functions produce the same truth tables as the ones we defined before, but their **computational behaviour** is different, which becomes important when proving properties about them.

5.3. Proving equations about Bool

Next, let's prove some interesting equalities. We are going to revisit our old friend, **distributivity** — but this time for booleans:

```
theorem distr_b : ∀ x y z : Bool,
  (x && (y || z)) = (x && y || x && z) :
intro x y z
cases x
. dsimp [and, or]
. dsimp [and]
```

After `intro x y z`, we are in the following state:

```
x y z : Bool
⊢ (x && (y || z)) = (x && y || x && z)
```

We do case analysis on `x`, which can be either

true or false. Hence, after cases `x`, we are left with two subgoals:

```
case false
y z : Bool
⊢ (false && (y || z)) = (false && y || t
case true
y z : Bool
⊢ (true && (y || z)) = (true && y || t
```

Let's discuss the first one. We can instruct Lean to use the definition of `&&` (i.e. and) by saying `dsimp [and]`. We are then left with:

```
y z : Bool
⊢ false = (false || false)
```

Now we just need to apply the definition of `||` (i.e. or) using `dsimp [or]` and we get `false = false` which is automatically discharged.

Can you see why in the second case it is enough to use `dsimp [and]`?

We can apply several definitions in one tactic; for example, in the first case `dsimp [and, or]` would have done the same. But actually, in this proof there is no need for `dsimp` at all because `rfl` automatically reduces its arguments. Hence we could have just written:

```
example : ∀ x y z : Bool,
  (x && (y || z)) = (x && y || x && z) :=
intro x y z
cases x
. rfl
. rfl
```

However, when doing proofs interactively it may be helpful to see the reductions. Also, later we will encounter cases where using `dsimp` (or its friend `simp`) is actually necessary.

Could we have used another variable than `x`?

Let's see — what about z ?

```
example : ∀ x y z : Bool,
  (x && (y || z)) = (x && y || x && z) := by
intro x y z
cases z
. sorry
. sorry
```

After `cases z`, we are stuck in the following state:

```
x y : Bool
⊢ x && (y || false) = x && y || x && false
```

No reduction is possible because we defined the functions by matching on the first argument. Using `cases y` would have done a bit of reduction but not enough. `cases x` was the right choice.

Now have a go at proving the **De Morgan** laws for `Bool` yourself. Both are provable just using case analysis:

```
theorem dm1_b : ∀ b c : Bool,
  (! (b || c)) = (! b && ! c) := by
theorem dm2_b : ∀ b c : Bool,
  (! (b && c)) = (! b || ! c) := by
```

5.4. Relating Bool and Prop

It may not have escaped your notice that we seem to define logical operations twice: once for `Prop` and once for `Bool`. How are the two related? We now define a function that maps `Bool` to the corresponding `Prop`

```
def isTrue : Bool → Prop
| b => b = true
```

`isTrue true` evaluates to `true = true` which is equivalent to `True` while `isTrue false` evaluates to `false = true` which is equivalent to `False`.

Using `isTrue` we can express the correctness of $\&\& \text{ wrt } \wedge$: namely `isTrue (b && c)` is equivalent to `isTrue b ∧ isTrue c`:

```

theorem and_ok : ∀ b c : Bool,
  isTrue (b && c) ↔ isTrue b ∧ isTrue c
intro b c
constructor
. intro H
  cases b
  . dsimp [and, isTrue] at H
    cases H
    . constructor
      . rfl
      . dsimp [and] at H
      assumption
. intro H
  cases H with
  | intro Hb Hc =>
    dsimp [isTrue] at Hb
    rw [Hb]
    dsimp [and]
    assumption

```

I am using `dsimp ... at ...` which simplifies a definition. Actually the proof also works without the `dsimps`.

The proof from left to right explores the truth table via case analysis while the proof from right to left uses equational reasoning. I recommend stepping through this proof interactively. We are only using tactics already explained. Note that `cases` is used in two different ways: for case analysis on booleans and to eliminate inconsistent assumptions.

I leave it as an exercise to prove the corresponding facts about negation and disjunction:

```

theorem or_ok : ∀ b c : Bool,
  isTrue (b || c) ↔ isTrue b ∨ isTrue c

theorem not_ok : ∀ b : Bool, isTrue (

```

5.5. No confusion from first principles

Previously we proved `true ≠ false` by doing cases on `true = false` and lean knows that this is impossible. However, we can actually prove this without this kind of magic and behind the curtains this is how the proof is actually implemented.

First of all we define an alternative version of `isTrue` but this time by pattern matching:

```
def isTrueD : Bool → Prop
| true => True
| false => False
```

Now we can prove no confusion just using rewriting:

```
theorem noConfBoolD : true ≠ false :=
  intro pcf
  have t : isTrueD true := by
    constructor
  rw [pcf] at t
  assumption
```

We know that we can prove `isTrueD true` but if `true = false` this can be rewritten to `isTrueD false` and this is `False` by definition.

← 4. Predicate Logic 6. The Natural Numbers →

IFR (COMP2065) lecture Search...

Table of Contents

- 1. Introduction
- 2. Propositional Logic
- 3. Classical Logic
- 4. Predicate Logic
- 5. The Booleans
- 6. The Natural Numbers
- 7. Lists
- 8. Trees

6. The Natural Numbers

- 6.1. Basic properties of $\mathbb{N}$
- 6.2. Structural recursion
- 6.3. Induction
- 6.4. Addition and its properties
- 6.5. Multiplication and its properties
- 6.6. Some Algebra
- 6.7. Ordering the numbers
- 6.8. Decidability

← 5. The Booleans

7. Lists →

6. The Natural Numbers

We have already used the natural numbers ($\mathbb{N}$) in examples but now we will formally define them. We will in spirit follow Giuseppe Peano who codified the laws of natural numbers using predicate logic in the late 19th century. This is referred to as **Peano Arithmetic**.



Giuseppe Peano

(1858–1932)

Peano viewed the natural numbers as created from zero ($0 = \text{zero}$) and `succ` (successor), i.e. $1 = \text{succ } 0$, $2 = \text{succ } 1$ and so on. In Lean this corresponds to the following inductive definition:

```
inductive Nat : Type
| zero : Nat
| succ : Nat → Nat
```

This declaration means:

- There is a new type `Nat` : `Type`,
- There are two elements `zero` : `Nat`, and given `n` : `Nat` we have `succ n` : `Nat`,
- All the elements of `Nat` can be generated by `zero` and then applying `succ` a finite number of times,
- `zero` and `succ n` are different elements,
- `succ` is injective, i.e. given `succ m = succ n` then we know that `m = n`.

We adopt the notation that $\mathbb{N} = \text{Nat}$, and Lean also automatically translates the usual decimal notation into elements of $\mathbb{N}$. This is convenient because otherwise it would be quite cumbersome to write out a number like `1234` : $\mathbb{N}$.

6.1. Basic properties of $\mathbb{N}$

6.1.1. No confusion

Let's verify some basic properties of $\mathbb{N}$ which actually correspond to some of Peano's axioms. First of all we want to verify that `succ n` $\neq$ `zero`, which corresponds to `true` $\neq$ `false` for `Bool` and indeed the same approach works here.

```
example :  $\forall$  n :  $\mathbb{N}$  , succ n  $\neq$  zero :=
intro n h
cases h
```

I leave it as an exercise to establish this **no confusion property** from first principles as we have done for `Bool`.

6.1.2. `succ` is injective

Next let's show that `succ` is injective. To do this we define a predecessor function `pred` using pattern matching, which works the same way as for `bool`:

```
def pred :  $\mathbb{N} \rightarrow \mathbb{N}$ 
| zero => zero
| succ n => n
```

We can test the function using `#reduce (pred 7)`.

We defined `pred 0 = 0` which is a bit arbitrary. However, `pred` does the job to show that `succ` is injective:

```
example :  $\forall m n : \mathbb{N}, \text{succ } m = \text{succ } n$ 
intro m n  $\square$  h  $\square$ 
change pred (succ m) = n  $\square$ 
rw [h]  $\square$ 
rfl  $\square$ 
```

Here we use `change` to replace the goal `m = n` with `pred (succ m) = n` exploiting that `pred (succ x) = x`. On the new goal we can apply `rw`.

However, there is also a tactic called `injection` which does this automatically for all inductive types, avoiding the need to define `pred`:

```
example :  $\forall m n : \mathbb{N}, \text{succ } m = \text{succ } n$ 
intro m n  $\square$  h  $\square$ 
injection h  $\square$ 
```

6.2. Structural recursion

When defining functions on $\mathbb{N}$ we will need recursion. Unlike the general recursion available in programming languages, we will only use **structural recursion**. That is, when we define a function on the natural numbers we can use the function on `n` to compute it for `succ n`. A simple

example is the `double` function, which doubles a number:

```
def double :  $\mathbb{N} \rightarrow \mathbb{N}$ 
| zero => zero
| succ n => succ (succ (double n))
```

Here `double (succ n)` is `succ (succ (double n))`. That is, every `succ` is replaced by two `succs`.

Because every natural number can be obtained by applying `succ` a finite number of times, recursive functions like this terminate in a finite number of steps. Non-terminating definitions, such as

```
def bad :  $\mathbb{N} \rightarrow \mathbb{N}$ 
| zero => zero
| succ n => succ (bad (succ n))
```

are rejected by Lean, since they don't define a mathematical function on $\mathbb{N}$.

We can define the inverse to `double` (division by 2 without remainder):

```
def half :  $\mathbb{N} \rightarrow \mathbb{N}$ 
| zero => zero
| succ zero => zero
| succ (succ n) => succ (half n)
```

Here `half` replaces every two `succs` by one.

6.3. Induction

Proof by induction is very closely related to structural recursion. It is basically the same idea, but for proofs. As an example, let's actually prove that `half` is the inverse of `double`:

```
theorem half_double :
   $\forall n : \mathbb{N}, \text{half (double n)} = n$  := by
intro n
induction n with
```

```

| zero =>  $\circ$  rfl  $\circ$ 
| succ n ih =>  $\circ$ 
  calc
    half (double (succ n))
    = half (succ (succ (double n)))
    _ = succ (half (double n)) := !
    _ = succ n := by  $\circ$  rw [ih]  $\circ$ 

```

After `intro n` we are in the following state:

```

n :  $\mathbb{N}$ 
 $\vdash$  half (double n) = n

```

`induction n` works very similarly to `cases`, but it gives us an extra assumption in the successor case—the **induction hypothesis**. That is in the `succ` case we have an extra argument (which I call `ih` for **induction hypothesis**).

```

m :  $\mathbb{N}$ 
ih : half (double m) = m
 $\vdash$  half (double (m + 1)) = m + 1

```

This means that we can assume the statement for `m` when proving it for `succ m` (which is displayed as `m + 1`.)

We prove this in several steps using `calc`, starting with

```
half (double (succ n))
```

we apply the definition of `double` (by saying `rfl`):

```
= half (succ (succ (double n))) := by
```

and now the definition of `half` (again using `rfl`)

```
_ = succ (half (double n)) := by rfl
```

The `_` is just an indicator for `lean` that we are chaining equations. Now we have a term that matches the induction hypothesis and we can just rewrite:

```
_ = succ n := by rw [ih]
```

and we are done.

This is a very simple inductive proof, but it shows the general idea. Because every number is finitely generated from `zero` and `succ`, we can **run** an inductive proof for any number by repeating the inductive step as many times as there are `succs`.

Induction is the main workhorse for proving properties of inductive types like $\mathbb{N}$. Try to fully understand the idea by looking at the simple proof above!

6.4. Addition and its properties

While addition is an operation which you may have learned already in kindergarten, it still needs to be defined. And, horror, its definition already uses recursion — they don't tell the kids this in kindergarten!

Here is Lean's definition of addition :

```
def add :  $\mathbb{N} \rightarrow \mathbb{N} \rightarrow \mathbb{N}$ 
| m , zero      => m
| m , (succ n) => succ (add m n)
```

`add m n` applies `n` successors to `m`. We define `m + n` as `add m n`. For example:

```
3 + 2
= add 3 2
= add 3 (succ 1)
= succ (add 3 1)
= succ (add 3 (succ 0))
= succ (succ (add 3 0))
= succ (succ (add 3 zero))
= succ (succ 3)
= 5
```

Lean also introduces the usual notation that `m +`

n is defined as `add m n`.

Lean defines addition by recursion over the second argument. While it might seem more natural to recurse over the first argument, the result is the same because addition is commutative ($m + n = n + m$), as we will show soon. We will stick with Lean's convention for now.

Now, what are the basic algebraic properties of $+$?

First of all, 0 is a **neutral element**, meaning $n + 0 = n$ and $0 + n = n$. You might think we only need to prove one of them since addition is commutative, but we actually need both when proving commutativity. It turns out that one is trivial, while the other requires induction.

The theorem `add_rneutr` ($n + 0 = n$) holds by definition of `add`.

```
theorem add_rneutr : ∀ n : ℕ, n + 0 =
intro n
rfl
```

However, `add_lneutr` ($0 + n = n$) does require induction.

```
theorem add_lneutr : ∀ n : ℕ, 0 + n
intro n
induction n with
| zero => rfl
| succ m ih =>
  calc 0 + (succ m)
      = succ (0 + m) := by rfl
      _ = succ m := by rw [ih]
```

This asymmetry comes from the definition of $+$ (add) by recursion over the 2nd argument. If we had defined it by recursion of the first it would be the other way around.

Another important property of addition is that brackets don't matter: $(1 + m) + n = 1 + (m + n)$.

$+ n$). This is called **associativity**. We again need induction, but the variable we choose for induction matters. Since addition is defined by matching on its second argument, we must perform induction on that argument (n).

```

theorem add_assoc :  $\forall l m n : \mathbb{N}$  , (1
intro l m  $\square$  n  $\square$ 
induction n with
| zero =>  $\square$  rfl  $\square$ 
| succ n' ih =>  $\square$ 
    calc (1 + m) + (succ n')
      = succ ((1 + m) + n') := by  $\square$ 
    _ = succ (1 + (m + n')) := by  $\square$ 
    _ = 1 + (succ (m + n')) := by  $\square$ 
    _ = 1 + (m + (succ n')) := by  $\square$ 

```

In the base case ($n = 0$), both sides reduce to $1 + m$. In the successor case, we are able to push through `succ` on both sides (using just the definition of $+$) to be able to apply the induction hypothesis.

We have now shown the following facts about $+$ and 0 :

- 0 is right neutral: $n + 0 = n$ (`add_rneutr`),
- 0 is left neutral: $0 + n = n$ (`add_lneutr`),
- $+$ is associative: $(1 + m) + n = 1 + (m + n)$ (`add_assoc`).

Such a structure is called a **monoid**. (If you look up "monoid" on Wikipedia, don't be distracted by the philosophical concept of the same name!)

However, we have not yet discussed **commutativity** ($1 + m = m + 1$), which isn't required for a monoid.

To prove commutativity, it isn't clear on which variable to do induction because both appear in

different positions. So we may as well do induction on the second (m) so that the left hand side reduces. The zero case can be proven using `l_neutr` which but the successor case is more difficult: our induction hypothesis is $ih : 1 + m = m + 1$ and we want to show $1 + \text{succ } m = \text{succ } m + 1$. We reason:

```
1 + (succ m)
  = succ (1 + m) := by rfl
```

and now we can already apply the induction hypothesis:

```
_ = succ (m + 1) := by rw[ih]
```

but how do we bridge the gap to `succ (m + 1)`? This is a common situation when proving theorems, which is a bit like using stepping stones to cross a stream. We need to put an additional stone in which in this case is a **lemma** (i.e. an auxiliary theorem), which can be proven by induction:

```
theorem add_succ :
  ∀ l m : ℕ, (succ l) + m = succ (l + m)
intro l m
induction m with
  | zero => rfl
  | succ m ih =>
    calc
      (succ l) + (succ m)
        = succ ((succ l) + m) := by
        _ = succ (succ (l + m)) := by
        _ = succ (l + succ m) := by
```

We can now use our **stepping stone** to prove commutativity:

```
theorem add_comm :
  ∀ l m : ℕ, l + m = m + l := by
intro l m
induction m with
  | zero =>
    calc l + 0
      = l := by rfl
```

```

      _ = 0 + 1 := by rw [add_one_neut]
| succ m ih =>
  calc
    1 + (succ m)
    = succ (1 + m) := by rw [rfl]
    _ = succ (m + 1) := by rw [add_comm]
    _ = (succ m) + 1 := by rw [add_comm]

```

The lemmas we needed to prove commutativity of addition are just the mirror image of the equation which follow from the definition, that is by definition we have that

$$\begin{aligned}
 m + 0 &= m \\
 m + (\text{succ } n) &= \text{succ } (m + n)
 \end{aligned}$$

and for commutativity we need to establish

$$\begin{aligned}
 0 + m &= m \\
 (\text{succ } m) + n &= \text{succ } (m + n)
 \end{aligned}$$

Together with the previous results, we have now established that $\mathbb{N}$ with $+$ and 0 forms a **commutative monoid**.

Mathematicians prefer it if you also have inverses as for the integers: for every integer $i : \mathbb{Z}$ there is $-i : \mathbb{Z}$ such that $i + (-i) = 0$ and $(-i) + i = 0$. Such a structure is called a **group**. Note that subtraction is now a derived operation $i - j = i + (-j)$.

6.5. Multiplication and its properties

We are proceeding slowly, and this time we will leave more of the fun to you. First, let's define the idea of multiplication informally.

As $+$ is repeated successor, $*$ is repeated addition. That is, $m*n$ is m added n times. For example, $3*2$ is $3 + 3 = 6$.

```

def mul :  $\mathbb{N} \rightarrow \mathbb{N} \rightarrow \mathbb{N}$ 
| _ , zero => zero
| m , (succ n) => mul m n + m

```

And as usual we define $x * y$ to stand for `mul x y`. As `+` was repeated `succ`, `*` is repeated `+`. That is $m * n$ is m added n times, for example::

```

3 * 2
= mul 3 2
= mul 3 (succ 1)
= mul 3 1 + 3
= mul 3 (succ 0) + 3
= mul 3 0 + 3 + 3
= 0 + 3 + 3
= 6

```

What are the properties of multiplication? First, it also forms a commutative monoid, with 1 now playing the role of 0 for addition — it is the neutral element for multiplication. The fundamental properties to prove are:

```

theorem mult_rneutr :  $\forall n : \mathbb{N}, n * 1 = n$ 
theorem mult_lneutr :  $\forall n : \mathbb{N}, 1 * n = n$ 
theorem mult_assoc :  $\forall l m n : \mathbb{N}, (l * m) * n = l * (m * n)$ 
theorem mult_comm :  $\forall m n : \mathbb{N}, m * n = n * m$ 

```

Proving these will certainly require the properties of addition that we have already established, and the order in which you prove them might differ from how they are listed here. You may also want to keep the next properties (distributivity and multiplication by zero) in mind as they interact with the proofs above.

Apart from addition and multiplication both being commutative monoids, they also interact in an important way known as **distributivity**. For example, to simplify an expression of the form $x * (y + z)$ you can “multiply out” to get $(x * y) + (x * z)$. There are also boundary cases with zero. Without assuming commutativity of multiplication, we want the symmetric statements

as well. The key properties are:

```

theorem mult_lzero :  $\forall n : \mathbb{N}, 0 * n$ 
theorem mult_rzero :  $\forall n : \mathbb{N}, n * 0$ 
theorem mult_ldistr :  $\forall l m n : \mathbb{N},$ 
theorem mult_rdistr :  $\forall l m n : \mathbb{N},$ 

```

The structure consisting of two monoids together with these distributivity laws (and with addition commutative) is called a **semiring**. When multiplication is commutative as well, we have a **commutative semiring**. Rings and semirings are central in algebra and closely related to polynomials (expressions like $7 * x^2 + x + 5$, possibly with higher exponents and several variables).

6.6. Some Algebra

Once we know that multiplication and addition form a commutative ring, we can establish many familiar identities. For example, the binomial formula $(x + y)^2 = x^2 + 2 \cdot x \cdot y + y^2$. To state this, we define exponentiation as repeated multiplication (just as multiplication is repeated addition).

In practice, proving polynomial identities by hand can be tedious. Lean's mathlib provides automation in form of a `ring` tactic that can solve any equality that follows from the ring or semiring axioms by normalising expressions to a canonical polynomial form and comparing them. The important point is that the tactic still produces a proof from first principles — there is no “cheating,” only verified calculation.

Here is a quick overview of standard number systems and their algebraic structures:

- Natural numbers ($\mathbb{N}$): **Semiring**

- Integers ($\mathbb{Z}$): **Ring**
- Rational numbers ($\mathbb{Q}$): **Field**
- Real numbers ($\mathbb{R}$): **Complete field**
- Complex numbers ($\mathbb{C}$): **Algebraically complete field**

A **ring** is a semiring with **additive inverses**: for every $x : \mathbb{Z}$ there is $-x : \mathbb{Z}$ such that $x + (-x) = 0$ and $(-x) + x = 0$. Subtraction is not primitive but is defined by adding the additive inverse: $x - y := x + (-y)$.

A **field** also has **multiplicative inverses** for all numbers different from 0. The simplest example is the rational numbers (fractions). For every $p : \mathbb{Q}$ with $p \neq 0$ there is $p^{-1} : \mathbb{Q}$ such that $p \cdot p^{-1} = 1$ and $p^{-1} \cdot p = 1$. For a fraction $(a/b) : \mathbb{Q}$ the multiplicative inverse is $(a/b)^{-1} = b/a$.

The real numbers include numbers like $\sqrt{2} : \mathbb{R}$ and $\pi : \mathbb{R}$, and have the additional property that any convergent infinite sequence has a unique limit. The complex numbers include $i = \sqrt{-1} : \mathbb{C}$ and have the additional feature that every polynomial equation has a solution (for example $x^2 + 1 = 0$); this is called **algebraic completeness**.

6.7. Ordering the numbers

Next we look at the relation $\leq$, which defines a **partial order** on the natural numbers. This time we are not going to use Lean's definition which uses an inductively defined relation (which we will cover later) but we define our own (equivalent) version.

We say that $m \leq n$ if there exists a number $k : \mathbb{N}$ such that $n = k + m$.

```
def LE : Nat → Nat → Prop
| m , n => ∃ k : ℕ , k + m = n

infix:50 (priority := 1001) " ≤ " =>
```

A **partial order** is a relation that is

- **Reflexive**: for all x , $x \leq x$,
- **Transitive**: for all x, y, z , $x \leq y$ and $y \leq z$ imply $x \leq z$,
- **Antisymmetric**: for all x, y , $x \leq y$ and $y \leq x$ imply $x = y$.

It is not hard to prove reflexivity and transitivity for the definition above (reflexivity with $k = 0$, transitivity by adding the “differences” and using associativity).

```
theorem refl_LE : ∀ n : ℕ, n ≤ n := λ
intro n
exists 0
apply add_lneutr

theorem trans_LE : ∀ l m n : ℕ, l ≤ m ∧ m ≤ n → l ≤ n
intro l m n
intro lm mn
cases lm with
| intro k klm =>
  cases mn with
  | intro j jmn =>
    exists j + k
    calc j + k + l
      = j + (k + l) := by rw
      _ = j + m := by rw [klm]
      _ = n := by assumption
```

Antisymmetry takes a bit more work and benefits from a few auxiliary lemmas — a good exercise.

```
theorem anti_sym_LE : ∀ l m : ℕ, l ≤ m ∧ m ≤ l → l = m
sorry
```

We can define $<$ by $m < n$ iff $m + 1 \leq n$.

```
def LT : ℕ → ℕ → Prop
| m , n => m+1 ≤ n

infix:50 (priority := 1001) " < " =>
```

I will not discuss $<$ in detail here: it is antireflexive (**for all** n , **not** $n < n$), transitive, and strictly antisymmetric ($m < n$ and $n < m$ implies `False`). Two key properties of $<$ are:

- **Well-foundedness:** any strictly descending chain (for example $10 > 5 > 3 > 0$) eventually terminates. This property underpins proofs of termination for algorithms that are not primitive recursive.
- **Trichotomy:** for any two numbers m , n , exactly one of $m < n$, $m = n$, or $n < m$ holds.

A relation that is transitive, trichotomous, and well-founded is called a **well-order**. Well-orders are very useful; there is a famous principle by Cantor, proven by Zermelo using the axiom of choice, that every set can be well-ordered. This is equivalent to the axiom of choice and not accepted in intuitionism (it implies excluded middle), and no explicit well-ordering of the real numbers is known.

6.8. Decidability

Equality for natural numbers is **decidable**: there is an algorithm that, given $m, n : \mathbb{N}$, determines whether $m = n$. This can be implemented as a function:

```
def eq_ℕ : ℕ → ℕ → Bool
| zero , zero => true
| zero , succ n => false
```

```

| succ m , zero => false
| succ m , succ n => eq_ℕ m n

```

We claim that $\text{eq_}\mathbb{N}$ decides equality that is $m = n \Leftrightarrow \text{eq_}\mathbb{N} \ m \ n = \text{true}$ for any $m \ n : \mathbb{N}$. We are now going to prove this. To show the left to right direction it is sufficient to show that $\text{eq_}\mathbb{N}$ is reflexive.

```

theorem refl_eq_ℕ : ∀ n : ℕ, eq_ℕ n n
intro n
induction n with
| zero => rfl
| succ n ih => calc
    eq_ℕ (n + 1) (n + 1) =
    eq_ℕ n n := by rfl
    _ = true := by rw [ih]

```

Using rewriting this implies the left to right implication:

```

theorem equal2eq : ∀ m n : ℕ, m = n -
intro m n mn
calc
    eq_ℕ m n
    = eq_ℕ m m := by rw [mn]
    _ = true := by rw [refl_eq_ℕ]

```

The left to right direction is more interesting. A first attempt fails:

```

example : ∀ m n : ℕ, eq_ℕ m n = true
intro m n mn
induction m with
| zero =>
    cases n with
    | zero => rfl
    | succ n' => cases mn
| succ m' ih =>
    cases n with
    | zero => cases mn
    | succ n' => sorry

```

We get into the following state

```

m' n' : ℕ
ih : eq_ℕ m' (n' + 1) = true → m' = n'
mn : eq_ℕ (m' + 1) (n' + 1) = true

```

$$\vdash m' + 1 = n' + 1$$

We need to apply `ih` for `n'` and not `n' + 1`. this is a common situation with induction proofs we have to make sure that our induction hypothesis is general enough. In this case this can be easily achieved by delaying the `intros` so that the induction hypothesis applies to all `n`.

```

theorem eq2equal :  $\forall m n : \mathbb{N}, \text{eq\_}\mathbb{N} m$ 
intro m
induction m with
| zero =>
  intro n mn
  cases n with
  | zero => rfl
  | succ n' => cases mn
| succ m' ih =>
  intro n mn
  cases n with
  | zero => cases mn
  | succ n' =>
    have h : m' = n' := by
      apply ih
      calc
        eq_ℕ m' n'
        = eq_ℕ (succ m') (succ
        = true := by rw [mn]
rw [h]

```

Now we can package everything up and prove that `eq_ℕ` decides equality:

```

theorem dec_eq_ℕ :  $\forall m n : \mathbb{N}, m = n \leftrightarrow$ 
intro m n
constructor
. apply equal2eq
. apply eq2equal

```

We say that equality of natural numbers is **decidable**. Not every predicate or relation is decidable, a famous example which we will see in the next semester is the **Halting problem**. However, there are simpler examples, eg equality on functions on natural numbers cannot be decided. A positive example is the relation $\leq$

which is decidable. I leave this as an exercise.

```
def le_N :  $\mathbb{N} \rightarrow \mathbb{N} \rightarrow \text{Bool}$ 
:= sorry
```

```
theorem dec_LE_N :  $\forall m n : \mathbb{N}, m \leq n \leftarrow$ 
:= by  $\square$  sorry  $\square$ 
```

← 5. The Booleans

7. Lists →

IFR (COMP2065) lecture Search...

Table of Contents

1. Introduction
2. Propositional Logic
3. Classical Logic
4. Predicate Logic
5. The Booleans
6. The Natural Numbers
7. Lists
8. Trees

7. Lists

- 7.1. Basic properties of `List`
- 7.2. Induction for Lists
- 7.3. The free monoid
- 7.4. Reverse
- 7.5. Insertion sort
- 7.6. Permutations

← 6. The Natural Numbers

8. Trees →

7. Lists

If you have already used a functional programming language like Haskell you know that lists are a very ubiquitous datastructure. Given $A : \text{Type}$ there is $\text{List } A : \text{Type}$ which is the type of finite sequences of elements of A . So for example $[1,2,3] : \text{List } \mathbb{N}$ or $[\text{true}] : \text{List } \text{Bool}$, we can also have lists of lists so for example $[[], [0], [0,1], [0,1,2]] : \text{List } (\text{List } \mathbb{N})$ is a list of lists of numbers. However, all elements of a list must have the same type, hence something like $[1, \text{true}]$ is not permitted.

For any type A we always can construct the empty list $[] : \text{List } A$, which is also called `nil` (this comes from the phrase **not in list** and it was used in the first functional language LISP which was first implemented in 1959). On the other hand if we have an element $a : A$ and a list $l : \text{List } A$ we can form the list $a :: l : \text{List } A$ with a in front of l , e.g. $1 :: [2,3] = [1,2,3]$. We call this `cons` - another heritage from LISP. Indeed the notation using $[\dots]$ is just a short-hand, e.g.:

$$[1,2,3] = 1 :: 2 :: 3 :: []$$

or using `nil` and `cons`:

$$= \text{cons } 1 (\text{cons } 2 (\text{cons } 3 \text{ nil}))$$

All lists can be created from `nil` and `cons` hence lists can be defined inductively in a manner very

similar to $\mathbb{N}$

```
inductive List(A : Type) : Type where
| nil : List A
| cons : A → List A → List A
```

This declaration means:

- For any type $A : \text{Type}$ There is a new type $\text{List } A : \text{Type}$,
- There are elements $\text{nil} : \text{List } A$, and given $a : A$ and $l : \text{List } A$ we have $\text{cons } a \ l$.
- All the elements of $\text{List } A$ can be generated by nil and then applying $\text{cons } a$ a finite number of times,
- nil and $\text{cons } a \ l$ are different elements,
- cons is injective, i.e. given $\text{cons } a \ as = \text{cons } b \ bs$ then we know that $a = b$ and $as = bs$.

7.1. Basic properties of List

Most of the basic properties and their proofs are very similar to the natural numbers. First of all we want to show that $[] \neq a :: l$ which corresponds to $0 \neq \text{succ } n$ and $\text{true} \neq \text{false}$ we had seen before. These properties are called **no confusion properties**. We can use cases as before for natural numbers:

```
theorem noConf :  $\forall a : A, \forall as : \text{List } A, [] \neq a :: as$ 
intro a as  $h$ 
cases  $h$ 
```

We can use the injection tactic to prove the injectivity of cons in both arguments:

```

theorem injCons_1 :  $\forall a b : A, \forall as l$ 
   $a :: as = b :: bs \rightarrow a = b := \mathbf{by}$ 
intro a b  $\square$  as  $\square$  bs  $\square$  h  $\square$ 
injection h  $\square$ 

theorem injCons_2 :  $\forall a b : A, \forall as l$ 
   $a :: as = b :: bs \rightarrow as = bs := \mathbf{by}$ 
intro a b  $\square$  as  $\square$  bs  $\square$  h  $\square$ 
injection h  $\square$ 

```

As before for $\mathbb{N}$ we can prove `injCons_2` from first principles using `tl` (tail)

```

def tl : List A  $\rightarrow$  List A
| [] => []
| _ :: as => as

theorem injCons_2' :  $\forall a b : A, \forall as$ 
   $a :: as = b :: bs \rightarrow as = bs := \mathbf{by}$ 
intro a b  $\square$  as  $\square$  bs  $\square$  h  $\square$ 
change tl (a :: as) = bs  $\square$ 
rw [h]  $\square$ 
rfl  $\square$ 

```

but it isn't so obvious what to do for `injCons_1` because we cannot complete the definition for `hd` (head):

```

def hd : List A  $\rightarrow$  A
| [] => sorry
| a :: as => a
end hd

```

There is no default element we can use to replace `sorry` - indeed the type `A` could be empty. I leave it as a challenge to prove `injCons_1` without using `injection`.

7.2. Induction for Lists

Induction for lists works very similar as for natural numbers: to prove a property for all lists `List A` we prove it for `[]` and we show that if it holds for a list `as` then it also holds for `a :: as` for any `a : A`.

We define the length function and append for lists and show by induction that the length of append is the sum of lengths. Ok let's start with length which is an easy recursion on lists:

```
def length : List A → ℕ
| [] => 0
| a :: as => succ (length as)
```

Now append is very similar to + but this time we do recursion on the first argument (why?).

```
def append : List A → List A → List A
| [] , as => as
| a :: as , bs => cons a (append as bs)
```

We adopt the usual notation writing `as ++ bs` for `append as bs`. Now we can state and prove the theorem that `length (as ++ bs)` is equal to `length as + length bs`.

```
theorem length_append : ∀ as bs : List A
  length (as ++ bs) = length as + length bs
intro as bs
induction as with
| nil =>
  calc length ([] ++ bs)
    = length bs := by rfl
  _ = 0 + length bs := by rw [add_zero]
  _ = length [] + length bs := by rw [length_nil]
| cons a as ih =>
  calc length (a :: as ++ bs)
    = length (a :: (as ++ bs)) :=
  _ = succ (length (as ++ bs)) :=
  _ = succ (length as + length bs) :=
  _ = succ (length as) + length bs :=
  _ = length (a :: as) + length bs := by rw [ih]
```

In the inductive step we are using the induction hypothesis `length (as ++ bs) = length as + length bs` to prove `length (a :: as ++ bs) = length (a :: as) + length bs`.

This would be easier if addition and append would be defined by recursion over the same argument but here we need to exploit the lemmas we

needed to show commutativity of addition.

Actually another approach would be to prove a different version of `length_append` which swaps the order and then use commutativity of `+` to derive the original theorem. I leave it as an exercise to complete the proof using list induction:

```
theorem length_append' :  $\forall$  as bs : List A
  length (as ++ bs) = length bs + length as
by  $\square$  sorry  $\square$ 
```

7.3. The free monoid

In the section on natural numbers we encountered a **monoid** using `+` and `0` on $\mathbb{N}$. There is a very similar monoid for lists using `++` and `[]`.

In the definition of `append` we are using pattern matching and structural recursion on lists. This works in the same way as for natural numbers: we can recursively use `append` for the smaller list `as`. To prove that this forms a monoid we need induction. However, different from `+`, now left neutrality is easy because it follows from the definition:

```
theorem app_lneutr :  $\forall$  as : List A, |
intro as  $\square$ 
rfl  $\square$ 
```

But now right neutrality requires induction:

```
theorem app_rneutr :  $\forall$  as : List A, |
intro as  $\square$ 
induction as with
| nil =>  $\square$ 
  rfl  $\square$ 
| cons a as ih =>  $\square$ 
  calc
    a :: as ++ []
  = a :: (as ++ []) := by rfl  $\square$ 
  _ = a :: as := by rw [ih]  $\square$ 
```

The switch between left and right neutrality is a consequence of that we have defined $+$ by recursion over the 2nd argument while we have defined $++$ by recursion over the first. We could have defined $+$ by recursion over the first argument and then this difference would have disappeared (I prefer this but the designers of the Lean standard library had different ideas). We couldn't have defined $++$ by recursion over the 2nd argument - do you see why?

The proof of associativity of $++$ again is very similar to the one for $+$. The main difference is that now we do induction over the first argument instead of the last - again this is caused by the fact that we now use recursion over the first instead of the 2nd argument.

```
theorem app_assoc :
  ∀ as bs cs : List A, (as ++ bs) ++ cs = as ++ (bs ++ cs)
intro as bs cs
induction as with
| nil =>
  rfl
| cons a as ih =>
  calc
    (a :: as ++ bs) ++ cs
    = a :: ((as ++ bs) ++ cs) := by
  _ = a :: (as ++ (bs ++ cs)) := by
  _ = a :: as ++ (bs ++ cs) := by
```

For $+$ we also proved commutativity that is $m + n = n + m$, but clearly this doesn't hold for $++$, e.g. $[1,2] ++ [3,4] = [1,2,3,4]$ while $[3,4] ++ [1,2] = [3,4,1,2]$.

Indeed $\text{List } A$ with $++$ and $[\]$ is the **free monoid** over A which intuitively means that only the monoid equations hold but no additional laws like commutativity. In this sense this monoid is free not to follow any laws apart from the monoid ones.

7.4. Reverse

Since the list monoid is not commutative, order matters. In particular we can **reverse** a list. That is we are going to define a function which given a list like $[1, 2, 3]$ produces the list with the elements in reverse order, in this case $[3, 2, 1]$.

How do we reverse a list? Recursively! The reverse of the empty list is the empty list and the reverse of a list of the form $a :: l$ is the reverse of l with a put in the end. So for example the reverse of $[1, 2, 3] = 1 :: [2, 3]$ is the reverse of $[2, 3]$, i.e. $[3, 2]$ with 1 put at the end giving $[3, 2, 1]$.

To define reverse we need an auxiliary operation which puts an element at the end. We call this operation `snoc` because it is the reverse of `cons`. We could define `snoc` using `++`, but for our purposes it is slightly more convenient to define it directly using recursion.

```
def snoc : List A → A → List A
| [], a => [a]
| a :: as, b => a :: (snoc as b)
```

Using `snoc` we can define `rev`:

```
def rev : List A → List A
| [] => []
| (a :: as) => snoc (rev as) a
```

A central property of `rev` is that it is self-inverse that is if we reverse a list twice we obtain the original list, e.g. $\text{rev} (\text{rev} [1, 2, 3]) = \text{rev} [3, 2, 1] = [1, 2, 3]$

Ok, let's try and prove this using list induction:

```
theorem revrev : ∀ as : List A , rev
intro as
```

```

induction as with
| nil => rfl
| cons a as ih =>
  calc
    rev (rev (a :: as))
      = rev (snoc (rev as) a) := by
    _ = a :: rev (rev as) := by sort
    _ = a :: as := by rw [ih]

```

I get stuck in the proof: it is clear that I need to apply the induction hypothesis but we need to know that $\text{rev } (\text{snoc } (\text{rev } as) a) = a :: \text{rev } (\text{rev } as)$. We are in the situation that we want to cross a stream but there are not enough stepping stones. To avoid getting our feed wet we need to put in the right stepping stone.

We cannot prove the equation we need directly but we can prove something more general by generalizing $\text{rev } as$ to any list. That is we need to prove $\text{rev } (\text{snoc } as a) = a :: \text{rev } as$ which is just the defining equation for snoc backwards. We can prove this by list induction:

```

theorem revsnoc :  $\forall a : A, \forall as : \text{List } A$ 
  rev (snoc as a) = a :: rev as := by
intro a as
induction as with
| nil => rfl
| cons b as ih =>
  calc
    rev (snoc (b :: as) a)
      = rev (b :: snoc as a) := by
    _ = snoc (rev (snoc as a)) b :=
    _ = snoc (a :: rev as) b := by
    _ = a :: rev (b :: as) := by

```

Using revsnoc we can complete revrev :

```

theorem revrev :  $\forall as : \text{List } A, \text{rev } (\text{rev } as) = as$ 
intro as
induction as with
| nil => rfl
| cons a as ih =>
  calc
    rev (rev (a :: as))
      = rev (snoc (rev as) a) := by

```

```

_ = a :: rev (rev as) := by rw
_ = a :: as := by rw [ih]

```

This is a nice example about the art of proving which is a bit like putting stepping stones into a stream to cross it without getting wet feet. When getting stuck with our induction, then looking at the point where we are stuck often leads us to identifying another property which we can prove and which helps us to complete the original proof. There is no fixed method to identify a good auxiliary property (a lemma) it is a skill which improves by practice.

Here is another problem to do with reverse which you can use to practice this skill: if you have attended Prof Hutton's Haskell course you will know that the above definition of reverse is very inefficient, indeed it has a quadratic complexity. A better definition is the following:

```

def revaux : List A → List A → List A
| [], bs => bs
| a :: as, bs => revaux as (a :: bs)

def fastrev : List A → List A
| l => revaux l []

```

fastrev only has linear complexity. However, we should convince ourselves that it behaves in the same way as rev, that is we should prove the following theorem:

```

theorem fastrev_thm : ∀ as : List A ,
  fastrev as = rev as := by
sorry

```

I leave it as an exercise to figure out what lemma(s) we need. Also it may be useful to employ some properties we have already proven.

7.5. Insertion sort

Finally, we are going to look at a slightly more interesting algorithm: insertion-sort. Insertion-sort is quite an inefficient algorithm because it has quadratic complexity while the best sorting algorithms have a $n \log n$ complexity. However, insertion-sort is especially easy to implement and verify.

To keep things simple I will only sort lists of natural numbers with respect to $\leq$: $\mathbb{N} \rightarrow \mathbb{N} \rightarrow \text{Prop}$ using the fact that we have an implementation in the form of `le_ℕ` : $\mathbb{N} \rightarrow \mathbb{N} \rightarrow \text{Bool}$ which decides $\leq$.

So our goal is to implement a function `sort` : `List ℕ → List ℕ` that sorts a given list, e.g. `sort [6,3,8,2,3] = [2, 3, 3, 6, 8]`.

Insertion sort can be easily done with a deck of cards: you start with an unsorted deck and you take a card from the top each time and insert it at the appropriate position in the already sorted pile. Inserting a card means you have to go through the sorted pile until you find a card that is greater and then you insert it just before that. OK, there is also the special case that the sorted pile is empty in which case you just make a pile out of one card. Here is the recursive function implementing the algorithm:

```
def insert : ℕ → List ℕ → List ℕ
| n , [] => [n]
| n , (m :: ms) =>
    match le_ℕ n m with
    | true => n :: m :: ms
    | false => m :: insert n ms
```

Using `insert` it is easy to implement `sort` by inserting one element after the other, starting with the empty list:

```
def sort : List ℕ → List ℕ
```

```
| [] => []
| (n :: ns) => insert n (sort ns)
```

7.5.1. Specifying Sorted

To verify that the algorithm indeed sorts we need to specify what it means for a list to be sorted.

That is we need to define a predicate `Sorted` : `List ℕ → Prop`. To define this we need to define an element relation for lists which we write `∈` to express that an element occurs in a list (e.g. `2 ∈ [1, 2, 3]`). This is similar to membership in a (finite) set and indeed in this is how membership is defined in the Lean library.

To define these predicates we are going to use inductive definitions of predicates which are similar to the inductive datatypes we have already seen. The basic idea is that we state some rules how to prove the predicate like we used constructors for inductive types like `zero`, `succ` or `nil` and `cons`.

7.5.2. Membership

Let's start with membership:

```
inductive Mem : A → List A → Prop
| mem_hd : ∀ {a : A}, ∀ {as : List A}
  -----
  Mem a (a :: as)
| mem_tl : ∀ {a b : A}, ∀ {as : List
  Mem a as →
  -----
  Mem a (b :: as)
```

We write `a ∈ as` for `Mem a as`.

The first constructor `mem_hd` (a rule without premises this is often called an axiom) tells us that

an element is a member of a list that starts with it, e.g. $2 \in [2, 3]$. The 2nd constructor `mem_tl` tells us that if we know that an element appears in the tail of a list then it also appears in the cons, eg. using $2 \in [2, 3]$ we can show $2 \in [1, 2, 3]$.

```
example : 2 ∈ [1,2,3] := by
  apply mem_tl
  apply mem_hd
```

Since this is an inductive definition we can also use cases and induction on `Mem`. For example we can show that there is no element in the empty list:

```
example : ∀ a : A , ¬ (a ∈ ([] : List
  intro a pcf
  cases pcf
```

Note that we have to tell Lean that we want to consider `[]` to be an element of `List A` with the same `a` we are using in the quantifier. `pcf` stands for **pigs can fly**. This works because there is no way to use `mem_hd` or `mem_tl` to prove that there is an element of the empty list.

We can also use cases to show that the elements of `[1,2]` are either 1 or 2:

```
example : ∀ n : ℕ, n ∈ [1,2] → n = 1
  intro n h
  cases h with
  | mem_hd =>
    left
    rfl
  | mem_tl h' =>
    cases h' with
    | mem_hd =>
      right
      rfl
    | mem_tl pcf =>
      cases pcf
```

Note that we also use `cases pcf` in the end to

show that there is no other possibility.

7.5.3. Sorted

We specify the predicate `Sorted` also as an inductive predicate.

```
inductive Sorted : List ℕ → Prop

| sorted_nil :
  -----
  Sorted []

| sorted_cons : ∀ {m : ℕ}, ∀ {ns : List ℕ}
  Sorted ns →
  (∀ n : ℕ, n ∈ ns → m ≤ n)
  -----
  → Sorted (m :: ns)
```

We specify that the empty list is sorted, and if you have a sorted list `ns` and you can show that all elements of `ns` are less than a new element `m` then also `m :: ns` is sorted.

As an example I show that `[1, 2, 3]` is sorted:

```
example : Sorted [1, 2, 3] := by
  apply sorted_cons
  . apply sorted_cons
    . apply sorted_cons
      . apply sorted_nil
      . intro n h
        cases h
    . intro n h
      cases h with
      | mem_hd =>
        apply le2LE
        rfl
      | mem_tl x => cases x
  . intro n h
    cases h with
    | mem_hd =>
      apply le2LE
      rfl
    | mem_tl hh =>
      cases hh with
      | mem_hd =>
```

```

      apply le2LE
      rfl
    | mem_tl hhh => cases hhh

```

I am using

```
le2LE : ∀ m n : ℕ, le_ℕ m n → m ≤ n
```

to establish facts like $1 \leq 3$. Indeed Lean defines a tactic `decide` which uses decidability to derive proofs. However, this doesn't work for our homegrown definitions.

Yes this is a rather laborious proof for such a trivial fact. Nobody would do this by hand and indeed it isn't hard to see that `Sorted` is decidable. But in the end we don't want to decide whether something is sorted but we actually want to sort!

7.5.4. Verifying insertion sort

Our goal is to verify that the output of insertion sort is a sorted list, that is we want to show:

```

theorem sort_sorts : ∀ ns: List ℕ, Sorted (sort ns)
sorry

```

It is not hard to see that we need to proceed by list induction and it is easy now to identify the main lemma about `insert`:

```

theorem sorted_insert : ∀ ns : List ℕ
  Sorted ns → Sorted (insert n ns) :=
sorry

```

Once we have established this lemma the rest is very straightforward:

```

theorem sort_sorts : ∀ ns: List ℕ, Sorted (sort ns)
intro ns
induction ns with
| nil =>
  apply sorted_nil
| cons n ns ih =>
  dsimp [sort]

```

apply sorted_insert $\square$
assumption $\square$

The real challenge is to show sorted_insert. I found it quite laborious and I don't want to spoil the fun. Here are some hints

- You will have to analyze the output of $\text{le_}\mathbb{N}$ $n\ m$ but just doing cases $\text{le_}\mathbb{N}$ $n\ m$ is useless, instead use cases $b : \text{le_}\mathbb{N}$ $n\ m$ which adds assumptions $b : \text{le_}\mathbb{N}$ $n\ m = \text{true}$ and $\text{le_}\mathbb{N}$ $n\ m = \text{false}$ in each of the cases.
- You need to show that $\neg m \leq n \rightarrow n \leq m$ i.e. that the order is linear.
- You need to prove some property of $\in$ here is a lemma I found useful:

$$\begin{aligned}
 & (\forall a : A, PP\ a \rightarrow QQ\ a) \\
 \rightarrow & \forall as : \text{List } A, \\
 & (\forall a : A, a \in as \rightarrow PP\ a) \\
 \rightarrow & \\
 & (\forall a : A, a \in as \rightarrow QQ\ a)
 \end{aligned}$$

where $PP\ QQ : A \rightarrow \text{Prop}$.

7.6. Permutations

Are we done now? Have we verified sort? That is, if you buy a tin and it says we have proven the theorem sort_sorts, are you then satisfied that the program sort does the job of sorting?

Not so! We have missed one important aspect of sort, namely that the output should be a rearrangement, i.e. a **permutation** of the input. Otherwise an implementation of sort that always returns the empty list would be OK.

We will specify the relation $\text{Perm} : \text{List } A \rightarrow$

List $A \rightarrow \text{Prop}$ which specifies that a list is permutation of another list, e.g. $\text{Perm } [1,2,2,3] \ [2,1,3,2]$ holds but $\text{Perm } [1,2] \ [1,3]$ or $\text{Perm } [1,2] \ [1,2,2]$ do not. Our goal is to prove

```
theorem perm_sort :
   $\forall ns : \text{List } \mathbb{N}, \text{Perm } ns \ (\text{sort } ns) :=$ 
sorry○
```

To define Perm we first define a relation $\text{Insert} : A \rightarrow \text{List } A \rightarrow \text{List } A \rightarrow \text{Prop}$ where $\text{Insert } a \ bs \ abs$ means that abs is obtained by inserting a into bs at an arbitrary position, e.g. we have $\text{Insert } 2 \ [1,3] \ [1,2,3]$.

```
inductive Insert :  $A \rightarrow \text{List } A \rightarrow \text{List } A \rightarrow \text{Prop}$ 
| ins_hd :  $\forall \{a:A\}, \forall \{as : \text{List } A\},$ 
  -----
  Insert  $a \ as \ (a :: as)$ 
| ins_tl :  $\forall \{a b:A\}, \forall \{as as' : \text{List } A\},$ 
  -----
  Insert  $a \ as \ as' \rightarrow$ 
  -----
  Insert  $a \ (b :: as) \ (b :: as')$ 
```

The idea is similar to Mem , we insert an element either in the beginning or somewhere in the tail. Here is a use case:

```
example : Insert  $2 \ [1,3] \ [1,2,3] := \mathbf{!}$ 
apply ins_tl○
apply ins_hd○
```

We now define Perm using Insert , basically the permutation of a fixed list can be obtained by inserting each element at an arbitrary position:

```
inductive Perm :  $\text{List } A \rightarrow \text{List } A \rightarrow \text{Prop}$ 
| perm_nil :
  -----
  Perm  $[] \ []$ 
```

```

| perm_cons : ∀ {a : A}, ∀ {as bs bs'
  Perm as bs →
  Insert a bs bs' →
  -----
  Perm (a :: as) bs'

```

Here is an example proving that $[3, 2, 1]$ is a permutation of $[1, 2, 3]$:

```

example : Perm [1,2,3] [3,2,1] := by
  apply perm_cons
  . apply perm_cons
    . apply perm_cons
      . apply perm_nil
      . apply ins_hd
    . apply ins_tl
      apply ins_hd
  . apply ins_tl
    apply ins_tl
    apply ins_hd

```

The proof that insertion sort produces a permutation is quite straightforward. First we show that `insert` actually inserts an element using the relation `Insert`:

```

theorem insert_inserts : ∀ ns : List
  Insert n ns (insert n ns) := by
  intro ns n
  induction ns with
  | nil =>
    dsimp [insert]
    apply ins_hd
  | cons m ms ih =>
    dsimp [insert]
    cases le_ℕ n m with
    | true =>
      change Insert n (m :: ms)
      apply ins_hd
    | false =>
      change Insert n (m :: ms)
      apply ins_tl
      assumption

```

Using this lemma we only need a straightforward list induction to show the main result:

```

theorem perm_sort :
   $\forall$  ns : List  $\mathbb{N}$ , Perm ns (sort ns) :=
  intro ns
  induction ns with
  | nil =>
    dsimp [sort]
    apply perm_nil
  | cons n ns ih =>
    dsimp [sort]
    apply perm_cons
    . apply ih
    . apply insert_inserts

```

What are the properties of Perm? It is an equivalence relation, indeed reflexivity is easy to show:

```

theorem refl_perm :  $\forall$  as : List A, Perm as as
intro as
induction as with
| nil => apply perm_nil
| cons a as ih =>
  apply perm_cons
  . apply ih
  . apply ins_hd

```

The other two properties, symmetry and transitivity, are a bit harder and require some lemmas. Hence I leave them as an exercise:

```

theorem sym_perm :  $\forall$  as bs : List A,
  Perm as bs  $\rightarrow$  Perm bs as := by sor

```

```

theorem trans_perm :  $\forall$  as bs cs ds :
  List A, Perm as bs  $\rightarrow$  Perm bs cs  $\rightarrow$  Perm as ds

```

Once you have proven them you may have a go at the following theorem:

```

theorem perm_tl :  $\forall$  a : A,  $\forall$  as bs :
  List A, Perm (a :: as) (a :: bs)  $\rightarrow$  Perm as bs

```

← 6. The Natural Numbers

8. Trees →

IFR (COMP2065) lecture Search...

Table of Contents

- 1. Introduction
- 2. Propositional Logic
- 3. Classical Logic
- 4. Predicate Logic
- 5. The Booleans
- 6. The Natural Numbers
- 7. Lists
- 8. Trees

8. Trees

- 8.1. Compiler correctness

← 7. Lists

8. Trees

Trees are everywhere, there isn't just one type of trees but many datatypes that can be subsumed under the general concept of trees. Indeed the types we have seen in the previous chapters (natural numbers and lists) are special instances of trees.

Trees are also the most dangerously underused concept in programming. Bad programmers tend to try to do everything with strings, unaware how much easier it would be with trees. You don't have to be a functional programmer to use trees, trees can be just as easily defined in languages like Python - see [my computerphile video](#)

In this chapter we are going to look at two instances of trees: expression trees and sorted trees.

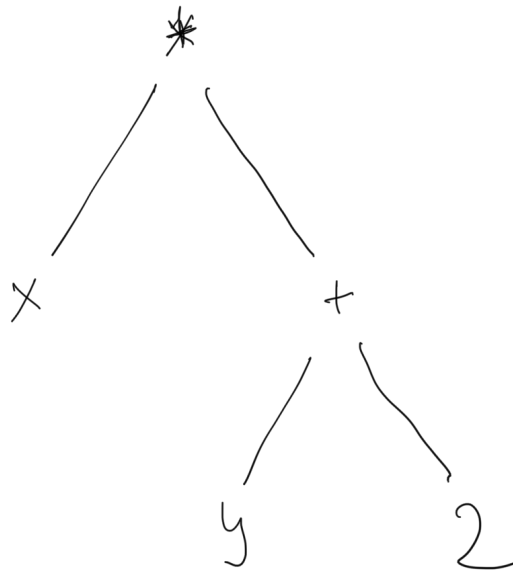
Expression trees are used to encode the syntax of expressions, we are going to define an interpreter and a compiler which compiles expression into the code for a simple stack machine and then show that the compiler is correct wrt the interpreter. This is an extended version of **Hutton's razor** an example invented by Prof Hutton.

The 2nd example is tree-sort: an efficient alternative to insertion-sort. From a list we produce a sorted tree and then we turn this into a sorted list. Actually, tree-sort is just quicksort in

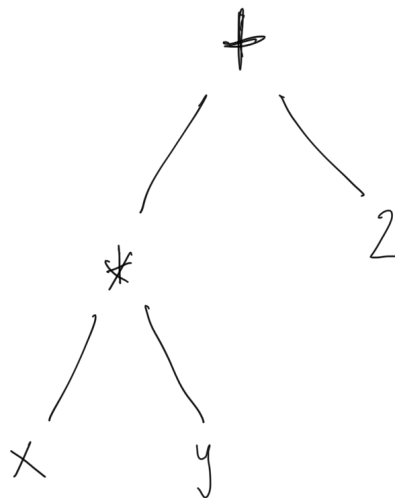
disguise.

8.1. Compiler correctness

I am trying to hammer this in: when you see an expression like $x * (y + 2)$ or $(x * y) + 2$ then try to see the tree. Really expressions are a 2-dimensional structure but we use a 1-dimensional notation with brackets to write them.



$x * (y + 2)$



$$(x * y) + 2$$

But it isn't only about seeing the tree we can turn this into a datatype, indeed an inductive datatype.

```
inductive Expr : Type
| const : ℕ → Expr
| var : String → Expr
| plus : Expr → Expr → Expr
| times : Expr → Expr → Expr
```

and we can translate our examples: $x * (y + 2)$:

```
def e1 : Expr
:= times (var "x") (plus (var "y") (c
```

and $(x * y) + 2$:

```
def e2 : Expr
:= plus (times (var "x") (var "y")) (
```

It is important to realize that there is no direct connection between elements of the type `Expr` and expressions in lean. While lean knows how to evaluate expressions:

```
def x : ℕ := 3
def y : ℕ := 5

#eval x * (y + 2)
#eval (x * y) + 2
```

it doesn't know how to evaluate `Expr`. If we try

```
#eval e1
#eval e2
```

all we get is the tree again (with names fully qualified).

Our first goal is to implement a specialized version of `#eval` which we call `eval`. It proceeds by recursion over the expression tree. To implement `eval` we need to resolve references to variables (which are our version of the heap). We do this by

defining a type `Env`:

```
def Env : Type
  := String → ℕ
```

We really should take care of variables which aren't defined (and use `Option`, which you may know as `Maybe` from Haskell) but we postpone run-time errors until later (refactoring) and adopt the very problematic convention that undefined is 0. Hence our test environment looks like this:

```
def env0 : Env
| "x" => 3
| "y" => 5
| _  => 0
```

The definition of `eval` now is straightforward proceeding by structural induction over the tree - translating every syntactical element with their meaning, variable are looked up in the environment and constants stand for themselves:

```
def eval : Expr → Env → ℕ
| const n , env => n
| var x , env => env x
| plus e1 e2 , env =>
  (eval e1 env) + (eval e2 env)
| times e1 e2 , env =>
  (eval e1 env) * (eval e2 env)
```

We test `eval` with our example expression and environment leading to the expected outcome:

```
#eval (eval e1 env0)
#eval (eval e2 env0)
```

We have defined an interpreter which provides a straightforward **denotational semantics** of our language. Next we provide a machine which can evaluate expressions. The machine is a stack machine which also accesses the environment which plays the role of a heap here. We define a type of instructions `Instr` and our `Code` is just a list of instructions.

```

inductive Instr : Type
| pushC :  $\mathbb{N}$   $\rightarrow$  Instr
| pushV : String  $\rightarrow$  Instr
| add : Instr
| mult : Instr

```

Our machine just has 4 instructions:

- `pushC n` : push `n` on the stack,
- `pushV x` : push the value of variable `x` into the stack,
- `add` : pop the two top items of the stack and replace them by their sum
- `mult` : pop the two top items of the stack and replace them by their product

We use `abbrev` to define `Code` so that we can use `++` on `Code`:

```

abbrev Code : Type
:= List Instr

```

Here is the hand translation of the first expression into stack machine code:

```

def c1 : Code
:= [pushV "x", pushV "y", pushC 2, add]

```

I leave it as an exercise to hand translate `e2`. To implement our machine we need a representation of a stack, we use lists again.

```

def Stack : Type
:= List  $\mathbb{N}$ 

```

We are now ready to define the **operational semantics**, i.e. the `run` function implementing our machine:

```

def run : Code  $\rightarrow$  Stack  $\rightarrow$  Env  $\rightarrow$   $\mathbb{N}$ 
| [] , [n] , env => n
| pushC n :: c , s , env =>
  run c (n :: s) env
| pushV x :: c , s , env =>

```

```

    run c (env x :: s) env
| add :: c , m :: n :: s , env =>
    run c ((n + m) :: s) env
| mult :: c , m :: n :: s , env =>
    run c ((n * m) :: s) env
| _ , _ , _ => 0 -- error

```

How does run work:

- if there is no code left, the stack should just contain the result which we return.
- in the other cases we dispatch on the first instruction:
 - if it is pushC or pushV we push the appropriate value on the stack.
 - if it is add or mult we assume that the arguments are on the top of the stack (note the order), pop them and replace them by the result.
- there are a number of error conditions, which we should signal explicitly (using Option or a more specific error type) but we still play quick and dirty and return 0.

Here is the first version of the compiler:

```

def compile : Expr → Code
| (const n) => [pushC n]
| (var x) => [pushV x]
| (plus l r) =>
    (compile l) ++ (compile r) ++ [ac]
| (times l r) =>
    (compile l) ++ (compile r) ++ [m]

```

The compiler translates every expression with a code that would leave the result of evaluating the expression on the stack. In particular for the binary operations it appends the code for the two subexpression and then issues the appropriate operation.

Let's test it on our examples:

```
#eval (run (compile e1) [] env0)
#eval (run (compile e2) [] env0)
```

The compile works fine and we could go on to prove correctness which is

```
theorem compile_ok :
  ∀ e : Expr, ∀ env : Env,
    eval e env = run (compile e) [] env
```

However, we will first reimplement the compiler using **continuation passing style** which is both more efficient and easier to verify because we avoid using ++ by passing the rest of the code (the continuation) as an argument. The idea is basically the same as for `fastrev`. It is easier to verify because we avoid ++ and so we don't have to refer to the monoid laws of `List`. Here is the continuation passing version of the compiler which uses an auxiliary function:

```
def compile_aux : Expr → Code → Code
| const n , c => pushC n :: c
| var x , c => pushV x :: c
| plus l r , c =>
  compile_aux l (compile_aux r (add))
| times l r , c =>
  compile_aux l (compile_aux r (mul))
```

In each of the cases we pass the code which comes after the current piece of code. The binary operations are translated into nested calls of `compile_aux`. We derive `compile` by supplying the empty code as the continuation:

```
def compile : Expr → Code
| e => compile_aux e []
```

To verify the compiler we need to prove a lemma which does the heavy lifting

```
theorem compile_aux_ok :
  ∀ e : Expr, ∀ env : Env,
  ∀ c : Code, ∀ s : Stack,
    run (compile_aux e c) s env
```

```

    = run c (eval e env :: s) env := by
intro e
induction e with
| const n =>
  intro env c s
  dsimp [compile_aux, run, eval]
| var x =>
  intro env c s
  dsimp [compile_aux, run, eval]
| plus l r ihl ihr =>
  intro env c s
  calc
    run (compile_aux (plus l r) c)
  = run (compile_aux l (compile_aux r c))
  = run (compile_aux r (add :: s))
  := by rw [ihl]
  = run (add :: c) (eval r env)
  := by rw [ihr]
  = run c ((eval l env + eval r env) :: s)
  := by rfl
  = run c (eval (plus l r) env)
| times l r ihl ihr =>
  intro env c s
  calc
    run (compile_aux (times l r) c)
  = run (compile_aux l (compile_aux r c))
  = run (compile_aux r (mult :: s))
  := by rw [ihl]
  = run (mult :: c) (eval r env)
  := by rw [ihr]
  = run c ((eval l env * eval r env) :: s)
  := by rfl
  = run c (eval (times l r) env)

```

The interesting bit is the use of tree induction. It is important to not introduce the quantifiers for c and s too early because they are changing during the proof. Note also the presence of two induction hypotheses for the binary constructors.

Using this lemma compiler correctness is now very straightforward:

```

theorem compile_ok :
  ∀ e : Expr, ∀ env : Env,
  eval e env = run (compile e) [] env
intro e env
calc
  eval e env

```

```

= run [] [eval e env] env := by
_ = run (compile_aux e []) [] env
_ = run (compile e) [] env := by

```

Admittedly this is a very simple language but it is sufficient to demonstrate the basic idea of compiler correctness. We could use this as a starting point for more realistic languages:

- we want to introduce some kind of branching via 'if-then-else'. A good approach is first to introduce a **tree structured code** which can then be translated into gotos in a separate step.
- we may want to define a simple type system, eg to distinguish numbers from booleans. Well typed code should avoid any run-time errors.
- we can also introduce functions using the stack and extending the type system.
- we may want also to implement loops or general recursion. The syntax for loops would also use tree-structured code but `eval` and `run` are problematic because a loop may fail to terminate. We can define a denotational semantics which allows non-termination using domain theory but this is a bit more challenging. A cheap solution is to supply a number of steps which limits the number of unfoldings and to define the operational semantics via a relation instead of a function.

← 7. Lists